Swarm Field

Final Report

by

Team members
Amartya Natayu
Abhronil Chaudhuri
Tycho Grapendaal
Jin Kuo
Błażej Łytkowski
Izzy van der Giessen

Document Revision:	Version 1.0
Case number:	5.3.2.
Coaches:	C. Verhoeven, P. Sundaramoorthy, R. Sabzevari and A. van der Niet
Company:	Robotics Institute
Project Duration:	08, 2026 - 11, 2026
Programme:	Joint Interdisciplinary Project

TU Delft JIP

Contents

1 Introduction **1**

2 Problem Definition and Sustainability Framework **2**

2.1 Problem Statement 2
2.2 Sustainability Goal 3
2.3 People, Planet, Prosperity 3

3 Proposed Concepts **5**

3.1 Shared Structure and Requirements . 5
3.2 Arbiter: Behavior Arbitration with Hysteresis 6
3.3 Concept 1: Multi-Layer Potential Field Superposition 6
3.4 Concept 2: Hop-Based Gradient Navigation 8
3.5 Concept 3: Hierarchical Quadtree Cost Propagation 9

4 Selection Procedure **12**

4.1 Analytic Hierarchy Process 12
4.2 Trade-off 13
4.3 Selected Solution 13

5 Implementation **15**

5.1 Final Result 15
5.1.1 Overview of the Solution . . 15
5.1.2 Memory 16
5.1.3 Behaviors 16
5.1.4 Arbiter 17
5.1.5 Role Rebalancing 17
5.1.6 Navigator and Potential Fields 18
5.1.7 Communication 18
5.1.8 Simulation Environment . . 19

6 Benchmark Design **20**

6.1 Compared Methods 20
6.2 Default Settings 20
6.3 Performance Measures 21
6.4 Running the Experiments 22
6.5 Experiments 22

7 Validation **24**

7.1 Experiments 24
7.1.1 Baseline (BASELINE) . . . 24
7.1.2 Exploration (EXPLORE) . . 25
7.1.3 Number of Scouts (NUM_EXPLORERS) . . . 26
7.1.4 Clearing Speed (SPEED) . . 28
7.1.5 Radio Range (RADIO) . . . 29
7.1.6 Scalability (SCALE) 30
7.1.7 Resupply (RESUPPLY) . . 33
7.1.8 Robot Failures (OUTAGE) . 35
7.1.9 Role Rebalancing (REBALANCE) 37
7.2 Discussion 38
7.2.1 Patterns Across the Experiments 39
7.2.2 The Methods Against the Requirements 39
7.3 Limitations of the Validation 39

8 Business Case and Deployment Potential **41**

8.1 Business Opportunity 41
8.2 Stakeholder Analysis 41
8.3 Commercial Model 42
8.4 Hardware Platform and Integration Scope 43
8.5 Software Development Costs 44
8.6 Revenue and Break-Even Analysis . 44
8.7 Potential Economic Value for Agricultural Users 46
8.8 Commercialization Requirements . 47

9 Risk, Ethics, and Mitigation Strategies **48**

9.1 Risk Management 48
9.1.1 Risk Assessment Method . . 48
9.1.2 Risks for the Robotics Institute 48
9.1.3 Impact and Risks for Society 50
9.1.4 Risk Matrix 51
9.2 Ethical Considerations 52
9.2.1 Ethical Criteria 52
9.2.2 Weighing the Design 52
9.2.3 Ethical Dilemmas 53

10 Interdisciplinary Teamwork **54**

10.1 Team Composition 54

10.2 Work Process 55
10.2.1 Division of Work 55
10.2.2 Tools and Agreements . . . 55
10.3 Collaboration with Stakeholders . . 55
10.4 Team Reflection 55
10.4.1 What Went Well 56
10.4.2 What Could Be Improved . 56
10.4.3 Lessons Learned 56

11 Conclusion and Recommendations 57
11.1 Conclusions 57
11.2 Recommendations 58

References 59

A Full Results of the Experiments 61

List of Abbreviations

AHP Analytic Hierarchy Process
AI Artificial Intelligence
FY Fiscal Year
ISO International Organization for Standardization
PPP People, Planet, and Prosperity
RTK-GNSS Real-Time Kinematic Satellite Positioning
SDG Sustainable Development Goal

1
Introduction

Robots are increasingly adopted in labor-intensive industries to address labor shortages, boost operational efficiency, and reduce cost (SHARFAEI, 2024; Graetz & Michaels, 2018). Agriculture is one such industry, where the robots are employed for a variety of tasks, including crop monitoring, harvesting, weeding, and pest treatment. Beyond these advantages, the agriculture robots can improve resource efficiency by minimizing water and chemicals usage and working round-the-clock (Vougioukas, 2019; Almtireen & Ryalat, 2026).

Existing agriculture robots are often designed for specific crops and may integrate multiple functions in one single platform. Although this approach can provide high task efficiency, it presents two major challenges: robustness and scalability. First, the agriculture robots are commonly relying on a centralized control system, creating a single point of failure that can compromise the operation of the entire system if critical component malfunctions (Jawhar, Mohamed, Wu, & Al-Jaroodi, 2018). Second, scaling up the number of robots is constrained by limitations in computational power, memory and communication bandwidth. Addressing these challenges through redundant hardware will increase the system complexity and costs.

To address these limitations, a decentralized swarm offers a promising alternative. Rather than relying on a central controller or individual robots capable of performing every task, a swarm consists of multiple robots with specialized roles that coordinate through local decision-making rules. These roles may include field exploration, pest treatment, harvesting, and charging, with some robots providing charging services to nearby robots. Furthermore, the robots will be able to switch between different configurations, and, when necessary, allow the swarm to adapt dynamically to changing field conditions and demands. By relying on local rules, the swarm can exhibit coordinated global behavior without a centralized controller. The decentralized system also addresses scalability, as additional robots can be incorporated without requiring exponentially growing computational, memory, and communication resources (Jawhar et al., 2018).

The architecture of the decentralized swarm is separated to three main components: behaviors module, arbiter, and navigator. The behaviors module evaluates the desirability of different tasks by assigning an importance score to each possible action. The developed model considers four behaviors: exploration, treatment, charging and resupplying. The arbiter uses the importance scores to select the robot's action and determine the transition between different behaviors and different targets. Finally, the navigator module guides the robot towards the target using one of two approaches depend on the configurations: hierarchical quadtree cost-propagation method and hop-based navigation. Together, these components provide a framework for coordinating robots through local rule, improving robustness and supporting scalability of the system.

This report begins with the problem definition and the sustainability framework developed by the team, as presented in Chapter 2. It then describes the concepts generated and the approach used to evaluate and select the final concept in Chapter 3 and Chapter 4, respectively. The subsequent chapters detail the implementation, benchmark design, and nine experiments conducted to validate the proposed solution, as presented in Chapter 5, Chapter 6, and Chapter 7. Consequently, Chapter 8 and Chapter 9 examines the potential business case, risk management, and ethical consideration of the product. Lastly, Chapter 10 reflects on the team's collaboration and interdisciplinary working process throughout the project.

2

Problem Definition and Sustainability Framework

This chapter defines the problem that the project addressed and places it in a broader sustainability context. Section 2.1 describes the background of the problem, the research objective, and the scope of the project. Section 2.2 links the project to the Sustainable Development Goals of the United Nations, and Section 2.3 translates these goals into People, Planet, and Prosperity aspects that guided the design.

2.1 Problem Statement

Pests and plant diseases cause large losses in global food production. For the five major food crops, these losses are estimated at $10\,\%$ to $40\,\%$ of the yield (Savary et al., 2019). Farmers limit these losses by monitoring their fields and by applying pesticides. Monitoring a large field by hand is labor-intensive, so an infestation may be found only after it has spread. Pesticides are therefore commonly applied uniformly to the whole field, often preventively on a fixed schedule, instead of only where they are needed: row crops are typically sprayed with broadcast sprayers across the entire field (Leise, Singh, Cafaro La Menza, Knezevic, & Jhala, 2025), and Dutch potato crops are sprayed against late blight 8 to 14 times per season (Kalkdijk, Evenhuis, & Schepers, n.d.). This harms the environment and conflicts with the European goal of halving the use and risk of chemical pesticides by 2030 (European Commission, 2020).

A swarm of small robots can monitor a field continuously and treat pests locally. In a swarm, each robot follows simple local rules and shares information with nearby robots, so that useful behavior emerges at the level of the swarm without a central controller. Such a decentralized system has no single point of failure and can, in principle, scale to large fields by adding robots. However, designing the local rules so that the swarm as a whole explores the field, detects pests, and allocates robots to treat them is not straightforward. Moreover, the performance of such an algorithm depends on the size of the swarm, on robot failures, and on how pests appear and spread, which makes it difficult to compare algorithms without a common benchmark.

The objective of this study was therefore to develop and explore, within ten weeks, a decentralized swarm-level algorithm that enabled a robot swarm to monitor and treat a virtual crop field. The swarm was to exhibit collective behavior through simple local rules, potential-field-based navigation, and peer-to-peer information sharing. The performance of the algorithm was benchmarked with predefined metrics, in order to identify the most effective approach for crop field monitoring. This objective led to the following main research question:

> How can a decentralized robot swarm, using only local rules, potential-field-based navigation, and peer-to-peer communication, detect and treat pests in a crop field efficiently and robustly?

The scope of the project was limited to the swarm algorithm and a simulation in which it was developed and evaluated. The physical robot platform, including the communication hardware and the sensors, was developed by the Swarm Head team and was not part of this project. The simulation represented the field as a coarse grid in which one cell covered a patch of field rather than a single plant. Each robot was assumed to know its own position, pests were static, and detection and treatment were assumed to be perfect. Within these assumptions, the algorithm was evaluated on open fields and fields

with rivers, with different swarm sizes, field sizes, pest rates and radio ranges, and with robot failures, resupply at a dock, and robots that switched roles during a run.

2.2 Sustainability Goal

The project was linked to the Sustainable Development Goals (SDGs) of the United Nations (United Nations, 2015). Three goals were identified to which a swarm for crop monitoring and treatment contributes most directly. Table 2.1 lists these goals, the specific target within each goal, and the contribution of the project.

Goal	Target	Contribution of the project
SDG 2: Zero Hunger	2.4: Ensure sustainable food production systems and resilient agricultural practices	Earlier detection and treatment of pests reduces yield loss
SDG 9: Industry, Innovation and Infrastructure	9.5: Enhance scientific research and upgrade technological capabilities	A benchmarked decentralized swarm algorithm for agricultural robotics
SDG 12: Responsible Consumption and Production	12.4: Achieve the environmentally sound management of chemicals	Local treatment instead of spraying the whole field reduces pesticide use

Table 2.1: Sustainable Development Goals and targets to which the project contributes.

The main sustainability goal of the project followed from these targets: to enable earlier and more targeted treatment of pests, so that less yield is lost while fewer pesticides are used. The project contributed to this goal indirectly. It did not deliver a deployed system, but an algorithm and a simulation that the Robotics Institute can use to develop such a system. The contribution to SDG 9 was therefore the most direct, while the contributions to SDG 2 and SDG 12 depend on a future deployment.

2.3 People, Planet, Prosperity

To make the sustainability goal concrete, it is translated into the three dimensions of People, Planet, and Prosperity (PPP), following the triple bottom line of (Elkington, 1997). For each dimension, the team identified what the swarm should achieve and how this was addressed in the design and the evaluation of the algorithm. Table 2.2 summarizes this translation.

Dimension	Aim	Addressed in the project by
People	Reduce repetitive monitoring work, operate safely and predictably around people	Arbiter with hysteresis for predictable behavior, robots that support rather than replace the farmer
Planet	Reduce pesticide use and prevent pests from spreading	Local treatment of pests, infested cells as the main performance measure
Prosperity	Reduce yield loss at an affordable cost, for small and large farms	Scalability metric, heterogeneous swarm of drones and ground robots, computationally efficient potential map

Table 2.2: Translation of the sustainability goal into People, Planet, and Prosperity aspects.

From the People perspective, the swarm is intended to support farmers and field workers by taking over the repetitive task of walking the field to look for pests. Since the robots operate close to people, their behavior has to be safe and predictable. This is one of the reasons for an arbiter with hysteresis, which keeps a robot on its current choice until another option is clearly better instead of changing direction whenever new information arrives. At the same time, automation changes the role of field workers and requires new skills for operating and maintaining the robots, which is discussed further in Section 9.1.3.

From the Planet perspective, the main aim is to reduce the use of pesticides. A swarm that detects an infestation early can treat it locally, before it spreads to the rest of the field and before treating the whole field becomes necessary. This aim is translated directly into the evaluation: the number of infested cells, averaged over a run, is the main performance measure (Section 6.3). It grows both with the number of untreated pests and with how long they stay untreated, so the swarm cannot score well by treating only the easy cases. The environmental cost of the robots themselves, such as batteries and electronic waste, is not part of the simulation and is addressed in Section 9.1.3.

From the Prosperity perspective, the swarm has to reduce yield loss at a cost that farmers can afford. The scalability metric measured whether adding robots improved the performance enough to justify their cost. The algorithm is designed for a heterogeneous swarm, in which drones explore the field and a smaller number of ground robots treat the pests, so that not every robot needs expensive treatment equipment. Finally, the potential map is computed on a quadtree data structure, which kept the computation affordable as the field grew and thus allows the same algorithm to be used on small and large farms.

3
Proposed Concepts

This chapter proposes three concepts for the swarm algorithm. All three share the same overall structure of the robot brain, but differ in what the robot stores in its memory and in how it navigates the crop field. This chapter first describes the shared structure and the requirements that a good concept has to meet, then the arbiter that all three concepts use in Section 3.2, and finally presents the three concepts in Section 3.3, Section 3.4, and Section 3.5.

3.1 Shared Structure and Requirements

For the following three concepts, we assume we have two types of robots: 1) killer robots that are big and slow, and move to flagged crops to treat them in some way, and 2) flying scout robots that are small and fast, used to monitor the field and notify the killer robots of flagged crops. In all three concepts, each robot decides for itself what to do next, using only its own memory: what it has sensed and what its peers have told it. Because much of this information arrives late or second-hand, it can be outdated or wrong. From its memory, the robot answers two questions: what to do, and which way to move. All robots answer these questions with the same components, so drones and ground robots differ only in their configuration.

A set of behaviors and an arbiter decide what to do. Each behavior is responsible for one activity, such as treating pests, exploring, or recharging, and proposes an action together with a importance score: how urgent that action is at the moment. The arbiter then selects one proposal. It has to do so without making the robot change course at every small change in its memory, which could result in robots looping or getting stuck more often.

A navigator using a potential map decides which way to move. The potential map assigns gradients at locations in the map such that moving downhill brings the robot to where its current action needs it, such as a pest or a charger. The navigator follows this slope one step at a time.

The concepts differ only in how they build the potential map and what information they build it with; the arbiter is the same in all three and is described in Section 3.2. Concept 1, multi-layer potential field superposition, sums one field per kind of information, such as obstacles, pests, and peers. Concept 2, hop-based gradient navigation, relays hop counts between robots to form a slope toward a pest. Concept 3, hierarchical quadtree cost propagation, spreads the value of each target outward over drivable ground. To compare the concepts, six requirements are set for the potential map:

Req. 1 One target: lead the robot to a single target, never to a point between two.
Req. 2 Obstacles and motion: respect obstacles and how the robot can move.
Req. 3 Never stuck: never trap the robot while a target is reachable.
Req. 4 Affordable: remain cheap to compute and store as the field grows.
Req. 5 One mechanism: work the same way for every behavior.
Req. 6 Collective: let robots coordinate, so that they divide targets among themselves instead of converging on the same one.

Chapter 4 compares the three concepts against these requirements.

3.2 Arbiter: Behavior Arbitration with Hysteresis

The arbiter decides what the robot is trying to achieve; the potential map only tells it how to get there. The arbiter sees nothing but the proposals of the behaviors and the clock, never the memory or a potential map, so it works unchanged with any set of behaviors and with each of the three maps. Each behavior submits a bid with an action, the target it commits the robot to, and a score between 0 and 1. All behaviors score on this one scale, so the same score means roughly the same urgency whichever behavior bids it: a score of 0 means that a behavior is considered useless, and a score of 1 means that all else should be dropped to do this behavior. A behavior scores its bid from the state and memory of the robot alone, so only the potential map of the winning behavior has to be built.

Because information arrives in bursts when robots meet, always taking the highest score would make the robots turn around constantly. The arbiter therefore holds the robot to its current commitment, a pair of an objective and a target, and applies three rules in order. First, a bid that scores at least 0.9 wins outright, whatever the commitment; this is reserved for needs that must not be traded off, such as a battery about to strand the robot. Second, a new objective is held for at least $30\,\mathrm{s}$. Third, after this period, a rival bid has to beat the current commitment by a margin

$$m = m_0 + wt, \tag{3.1}$$

where t is the time already spent on the current target as a fraction of $5\,\mathrm{min}$, capped at 1, and $w = 0.375$ the weight of this effort. The base margin m_0 is 0.125 for a rival objective and 0.0625 for another target of the same objective, because swapping one pest for another is a smaller decision than giving up treating for exploring. A robot that has just set off is thus easily redirected, while a robot that has spent five minutes on its target needs a rival that scores 0.5 higher, which from any commitment above 0.4 only an override can provide. Effort is measured in time rather than as the fraction of the journey completed, so the arbiter needs only the clock and not the length of the route. When no behavior bids for the committed objective any more, for example because the last known pest has been treated, the highest score wins at once; when only the committed target is gone, the robot moves on to the best target of the same objective.

In the first design of the arbiter, each objective scores its bid in cells from its own potential field, the field of Concept 3, as $s_o = u_o - \Phi_o(c)$, with u_o its urgency and $\Phi_o(c)$ its potential at the cell c of the robot. The margins are then 5 cells to change objective, 2.5 cells to change target, and 15 more cells near the end of a journey. On the shared scale these become the values above: in that design a logistic turns a potential into a score, and at its steepest point one cell is worth 0.025. Scoring from memory instead removes the need to build every field before a decision can be made.

The arbiter is greedy and unaware of other robots, so dividing targets among the robots is left to the potential map. A bid that follows a potential map carries no target, since the map picks the target only after the bid has won, so keeping a robot on one pest within a map is left to the map as well. Such a commitment also builds no effort, so its margin stays at the base value: counting the time spent on the objective instead would lock an exploring robot in. The parameters of the arbiter are initial estimates that still have to be tuned in simulation.

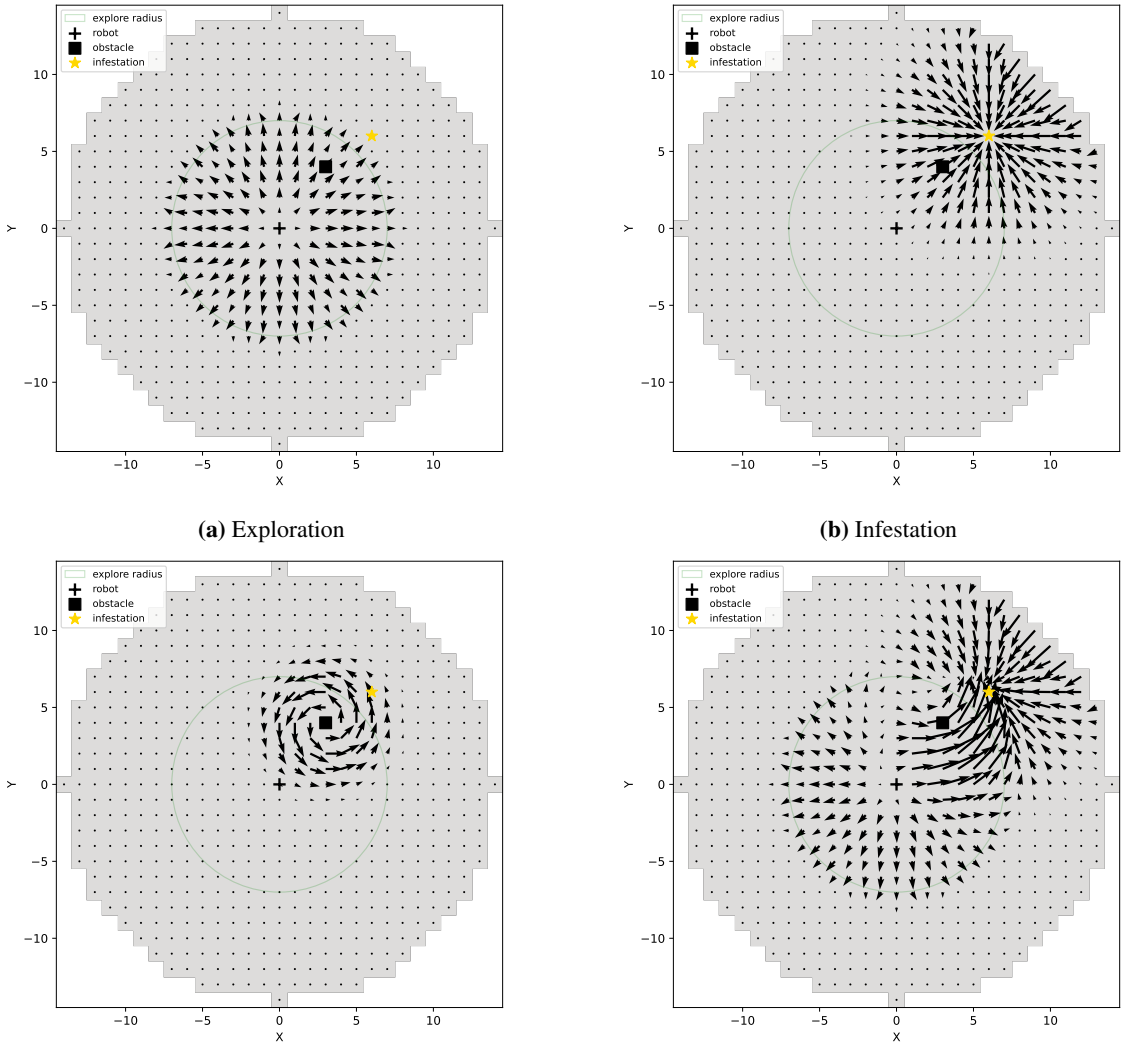
3.3 Concept 1: Multi-Layer Potential Field Superposition

The first concept combines local sensor observations with short-term spatial memory in a set of potential fields, one for each kind of information. The fields are superimposed into a single force field that guides the movement of the robot, while the arbiter of Section 3.2 decides which objective the robot pursues.

Each robot maintains several scalar potential fields, each representing one aspect of the environment: exploration history, infestation, obstacles, peers, and, eventually, crop information. Each field encodes the local influence of its environmental factor around the robot.

The exploration field acts as a persistent short-term memory. Each observation of a cell raises its exploration value by one, up to a maximum of 4, which allows the robot to distinguish recently explored areas from less explored ones. The other observations each generate their own response. An infestation generates a negative Gaussian field, whose depth and width scale with the severity of the observation and whose depth also scales with its confidence. Because the force is the negative gradient of the potential, this field attracts the robot toward the infestation. An obstacle generates a positive Gaussian field with a standard deviation of one cell, whose resulting force is rotated by 90 degrees, so that the robot is steered along the obstacle rather than pushed straight away from it. Peers generate a weaker positive Gaussian field of the same width, which allows nearby robots to influence the exploration behavior.

The potential fields are converted into force fields by estimating their local spatial gradients. The forces of the individual layers are then summed cell by cell into a net force field. Because the individual force maps are kept alongside the net field, the contribution of each environmental factor can be inspected and tuned independently. The last step, converting the net force field into discrete movement actions and scored bids for the arbiter, is left to the decision layer and is not yet implemented. Figure 3.1 presents the potential force maps of a robot with different observations.



(a) Exploration

(b) Infestation

(c) Obstacle

(d) Superposition

Figure 3.1: Illustration of potential force map with **(a)** a repulsive force from the observed region, **(b)** an attractive force from infestation, **(c)** a rotated force steering the robot away from an obstacle, and **(d)** all maps superimposed into a final force map.

3.4 Concept 2: Hop-Based Gradient Navigation

The second concept is a hop-based navigation strategy. What makes this concept unique is the way a killer robot reaches a detected plant in need of help, the target, without any robot sharing where that target lies. Every robot broadcasts a beacon at every decision: a hop number, its cell, and the time it is sent. A scout that finds a target becomes its source and broadcasts the hop number 0 on the cell of the target. It does not wait there but flies on, and keeps broadcasting 0 for a few minutes, which carries the target to robots further away. Every other robot is a relay: it gives itself the lowest hop number it hears plus one hop, and broadcasts that number on its own cell. The hop number of a robot thus counts the hops needed to pass a message from one robot to the next until the source is reached, and the numbers descend toward the target. When the plant has been treated, or the source no longer marks it, the hop number 0 is no longer broadcast, and the robots forget the numbers that were built on it.

A killer robot does not need to know where the pest is; it walks down the hop numbers it hears. Every robot remembers the newest beacon from each sender, and a beacon counts only for a fixed time after it is sent. The robot takes as its own hop number the lowest hop number it hears plus the cost of the hop to that beacon,

$$h = \min_{b \in H} \left(h_b + \frac{\max(d_b, d_{\min})}{R} \right), \tag{3.2}$$

where H is the set of heard beacons, h_b the hop number of beacon b, d_b the distance to its cell, R the radio range, and $d_{\min}$ one cell. A hop thus costs its length in radio ranges, so a beacon at the edge of the radio range adds about one hop and a beacon halfway adds about half. The minimum of one cell keeps two robots on the same cell from handing each other the same value forever.

Only the beacons with a hop number below the robot's own, the downhill beacons D, pull the robot. Each pulls toward its cell with a strength $s_b = \gamma^{h_b} e^{-a_b/\tau}$, where $\gamma = 0.5$ halves the pull with every hop, a_b is the age of the beacon, and τ its decay time. The robot walks in the direction of the summed pull

$$\mathbf{p} = \sum_{b \in D} \sigma_b \frac{s_b}{\max(d_b, 1)^k} \, \hat{\mathbf{u}}_b, \tag{3.3}$$

where $\hat{\mathbf{u}}_b$ is the unit vector from the robot toward beacon b. The sign σ_b is $+1$ for a beacon of a scout and -2 for a beacon of another killer robot, so a killer robot that is already closer to a pest pushes the others away, and the robots spread over the pests instead of all walking to the same one. Dividing by the distance to the power $k = 0.5$ keeps a robot between two pests from being held at the point where their pulls cancel. Figure 3.2 shows how the pulls of the individual beacons add up. The bid of the treating behavior is the strength of the strongest downhill beacon, so a robot commits to a pest only when something pulls it, and the bid vanishes when nothing does.

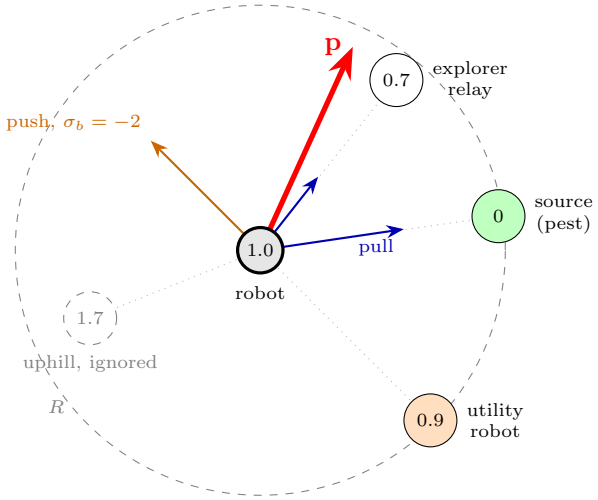


Figure 3.2: The summed pull on a killer robot in Concept 2. Each circle is a fresh beacon the robot remembers, labeled with its hop number from Equation 3.2, and the dashed circle is the radio range R. The source beacon on the pest gives the robot a hop number of about 1.0. The source and a scout relay closer to the pest pull the robot toward their cells. Another killer robot, which is closer to the pest, pushes it away with twice the weight. The beacon with hop number 1.7 is uphill and is ignored. The thick arrow is the summed pull $\mathbf{p}$ of Equation 3.3, and all arrows are drawn to the same scale.

The direction is re-evaluated at every decision from the beacons the robot remembers, so no map or waypoints are stored. The pull is handed to a steerer as a heading toward the cell of the strongest beacon. Ground-based killer robots cannot enter obstacle cells such as rivers and rocks: when the cell ahead is blocked, the steerer follows the obstacle on its right-hand side, using only the cells the robot senses around itself, until it can head toward the beacon again. A robot that makes too little progress over ten minutes counts as stuck and places a private escape beacon on the place it circles around, which pushes it away from there.

No robot announces that a pest has been treated. Instead, a robot that sees the pest gone stops broadcasting hop 0 at once, and the beacons further up the chain expire. Relays that still hear each other keep passing on values that rise with every round, so beacons above a cap of about 4.3 hops, where the pull falls below 5 percent, are ignored and not relayed, and these leftover values disappear.

3.5 Concept 3: Hierarchical Quadtree Cost Propagation

The third concept computes a map that tells a robot, at every location of the field, which target to head for and which way to step. The name of the concept has two parts: cost propagation is the idea, and the hierarchical quadtree is what makes it computationally efficient.

Each target starts with a head start equal to its value and loses a little of it for every cell traveled, like ripples spreading over a lake. Where the ripples of two targets meet, the better offer wins. Every cell c thus receives the potential

$$\Phi(c) = \min_i \left(d(c, s_i) - \beta v_i \right), \tag{3.4}$$

where s_i is the cell of target i, v_i its value, $d(c, s_i)$ the length in cells of the shortest route from c to s_i that the robot can actually drive, and β an exchange rate, such that target i is worth a detour of βv_i cells. The robot simply steps to the neighboring cell with the lowest potential.

The potential is computed as a wave that spreads outward from all targets at once. Each target cell starts with the potential $-\beta v_i$, so a more valuable target starts lower. From there, the wave moves to the neighboring cells, and each neighbor receives the potential of the cell it was reached from plus the length

of the step: 1 for a straight step and 1.4 for a diagonal one. Cells that the robot cannot enter, such as cells outside the field and, for a ground robot, obstacles such as a river, are never entered, so the wave flows around them; a diagonal step is allowed only when both cells beside it are free, so the robot never cuts the corner of an obstacle. A cell that is reached by several waves keeps the lowest potential and remembers the target that offered it. The wave always grows from the lowest potential found so far, so every cell is final the first time it is reached, and each cell has to be computed only once. For example, with target A worth a detour of 45 cells and target B worth 10 cells, a cell 50 cells from A and 20 cells from B receives the potential $50 - 45 = 5$ from A and $20 - 10 = 10$ from B. The robot at that cell thus heads for A, although B is closer, because the detour of 30 extra cells is worth less than the difference in value of 35 cells.

Several useful properties follow from Equation 3.4. Because it takes the best offer rather than a sum, every cell leads to exactly one target, and the field splits into territories, as shown in Figure 3.3. A valuable target claims a larger territory, which reaches past the midpoint toward a less valuable one. Distances are measured along drivable routes, so obstacles bend the field instead of pushing the robot away, and a drone and a ground robot obtain different fields from the same targets. Finally, every cell from which a target can be reached has a neighbor with a lower potential, namely the cell the wave reached it from, so a robot that follows the field can never be trapped in a local minimum, the classic failure of summed force fields (Koren & Borenstein, 1991). This holds exactly for the cells near the robot; further away, route lengths through merged blocks are approximate, and the map is built over the terrain the robot believes rather than the true terrain. A robot that stops making progress along the map therefore falls back to following the obstacle.

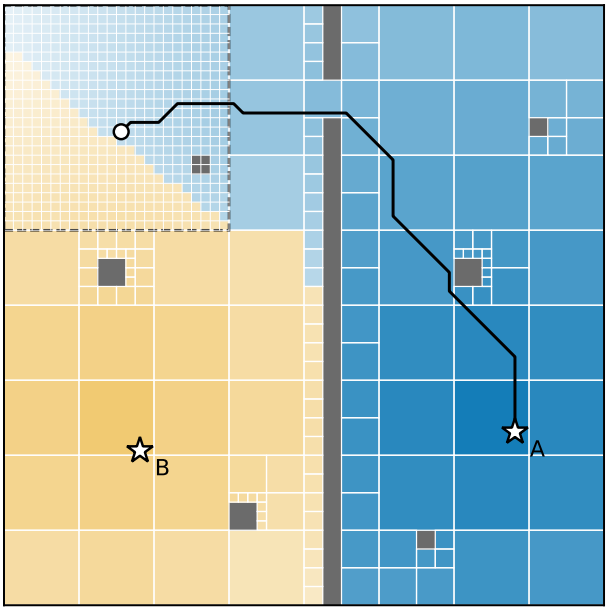


Figure 3.3: A field with two targets, computed with Concept 3 on a field of 64×64 cells. The color shows which target a robot at that location heads for, and a darker color means more attractive. Target A is worth a detour of 45 cells and target B of 10 cells, so the territory of A reaches through the gap in the wall (gray) into the western half. The white outlines are the blocks that the computation works on, and inside the dashed window around the robot (circle) every cell is computed exactly. The black line is the route of the robot.

A behavior only decides what is worth reaching and how much; the field works out how to get there. The treating behavior places targets on known pests, the charging behavior places them on the docks, and the battery robots place them on peers that report a low battery. The field is built from the memory of the robot itself, so no chain of robots leading to the target has to stay alive. Furthermore, targets that a peer is already heading for can be given a lower value, which spreads the robots over the pests instead of sending them all to the nearest one.

Computing the potential of every cell of a large field is expensive, and mostly unnecessary: the robot rebuilds the map whenever it enters a new block of 16×16 cells, so only its next steps have to be exact, while a rough direction suffices for distant parts of the field. Open ground is therefore merged into square blocks of up to 16×16 cells, a structure known as a quadtree, while cells remain single near obstacles and in a window of 80×80 cells around the robot. The potential map is filled by a single run of Dijkstra's algorithm on the quadtree, which is the wave described above, spreading from all targets at once. Table 3.1 compares this approach with computing every cell exactly on a field of 500×500 cells. The quadtree searches twenty times fewer places and is eight times faster. On fields of up to 120×120 cells, a robot that follows the quadtree map always reaches its target, along a route at most 2 percent longer than the shortest route. The advantage grows with the size of the field: on a field of 100×100 cells, the quadtree is 2.5 times faster.

Method	Places searched	Time per map	Route length
Every cell	250 000	2.1 s	Shortest
Quadtree	13 000	0.26 s	Within 2 percent

Table 3.1: Computing one potential map on a field of 500×500 cells with a river and 500 scattered obstacles, exactly and with the quadtree. The route length is measured on fields of up to 120×120 cells.

4
Selection Procedure

This chapter describes how one concept was selected from the three concepts of Chapter 3 to be developed into the final solution. To systematically identify the most suitable concept, a structured trade-off analysis was conducted. Section 4.1 explains how the weight of each requirement was assigned. Following that, Section 4.2 compares the potential maps of the three concepts and supports the comparison with technical arguments. Finally, Section 4.3 summarizes the chosen solution and the drawback that was accepted with it.

4.1 Analytic Hierarchy Process

The weights of the requirement were computed using Analytic Hierarchy Process (AHP), which evaluated their importance through pairwise comparison. The relative importance of one requirement over another was rated on a scale from 1 to 9: 1 mean equally importance, 5 mean strongly more important, and 9 mean extremely more important. Intermediate integer values were used to represent intermediate levels of importance.

Scalability and swarm behavior were considered the two most important requirements, as they represent the core objectives of the project. Obstacle avoidance and the ability to avoid get trapped were the next highest level of importance, as they directly influence the swarm's ability to operate successfully. The requirements concerning of single target and single mechanism were considered less critical, as failing to meet them was less likely to compromise the success of the swarm. Table 4.1 shows the pairwise comparison matrix of all requirements. The matrix is reciprocal, meaning that each entry represents the importance of the requirement in its row relative to the requirement in its column.

	Req. 1	Req. 2	Req. 3	Req. 4	Req. 5	Req. 6
Req. 1	1	$\frac{1}{5}$	$\frac{1}{5}$	$\frac{1}{7}$	3	$\frac{1}{9}$
Req. 2	5	1	1	$\frac{1}{3}$	5	$\frac{1}{5}$
Req. 3	5	1	1	$\frac{1}{3}$	7	$\frac{1}{2}$
Req. 4	7	3	3	1	7	1
Req. 5	$\frac{1}{3}$	$\frac{1}{5}$	$\frac{1}{7}$	$\frac{1}{7}$	1	$\frac{1}{7}$
Req. 6	9	5	2	1	7	1

Table 4.1: Analytic Hierarchy Process pairwise comparison matrix.

Subsequently, the matrix was squared, and the value in each row were summed. The weight of each requirement was then calculated by dividing its row sum by the sum of all row sums. Table 4.2 presents the resulting weight of each requirement, rounded to the nearest 0.05.

Requirement	Weight	Requirement	Weight
Req. 1: One target	0.05	Req. 4: Affordable	0.30
Req. 2: Obstacles and body	0.10	Req. 5: One mechanism	0.05
Req. 3: Never stuck	0.15	Req. 6: Collective	0.35

Table 4.2: Weight of each requirement.

4.2 Trade-off

All concepts were compared against the six requirements for a potential map that were defined in Chapter 3. For each requirement, the team assessed whether a concept fulfilled it fully, partly, or not at all, and supported this assessment with an argument or a calculation. To make the comparison explicit, the assessments were converted into scores ranging from 1 to 5: score of 1 indicated requirement was unlikely to met, score of 3 indicated a risk that the requirement would not be met, and a score of 5 indicated that the requirement was expected to be fully met. Table 4.3 shows the assessment of the three concepts against the requirements, together with the resulting scores and the arguments.

Requirement (Weight)	Concept 1	Concept 2	Concept 3
One target (0.05)	Exact target is not given; only a direction is given (1)	Lead the robot towards nearest or desired target/hob (5)	Cost propagation only provides best target (5)
Obstacles and body (0.10)	The potential force steer the robot away from the peers and obstacle (4)	Follow the hop gradient, extra mechanism need to be implemented to avoid collision (2)	Only drivable route will be provided (5)
Never stuck (0.15)	Virtual force or random movement need to introduce to prevent becoming trapped (3)	No mechanism to prevent becoming trapped (1)	Drivable route unlikely navigate to (4)
Affordable (0.30)	The computational power and memory grow with data (1)	Only hop number propagate through the network (5)	The memory grows with data (2)
One mechanism (0.05)	One layer per factor (5)	Require different mechanism for exploration behavior (2)	One target per behavior (5)
Collective (0.35)	Sharing information to have a better realization on environment; still be able to work independently (2)	The swarm need to collaborate together to achieve the goal (5)	Sharing information to improve efficiency; still be able to work independently (3)
Weighted score	2.15	3.95	3.25

Table 4.3: The three concepts against the requirements of Chapter 3, with the points per requirement in parentheses and the weighted score.

4.3 Selected Solution

As shown in Table 4.3, Concept 2, hop-based gradient navigation, scored highest in the trade-off, mainly for its small memory and its collective behavior. However, it lacks a mechanism for exploration. Concept 3, hierarchical quadtree cost propagation, ranked second, but its strength lay elsewhere. A robot keeps the pests that the swarm shares in its own memory, so it can act on a pest even when the robot that found it has moved away or failed. It needs no chain of robots that stays in contact.

Both concepts were therefore carried forward, as two competing ways for the killers to reach the pests. The hop-based killers follow Concept 2, and the seeded killers plan their routes with Concept 3. Concept 3 also provides the exploration for both, so the scouts are the same in either swarm. The quadtree keeps the computation of its map affordable (Table 3.1), which answers its low score for affordability.

The benchmark of Chapter 7 then decides between the two methods on the same fields.

Two drawbacks remained after this choice. First, Concept 2 had no mechanism to keep a killer clear of walls and other robots, or to free it when it was trapped. The final solution adds both: hop-based killers push each other away, and a trapped hop-based killer places an escape beacon that pushes it out (Section 5.1.6). Second, every robot must know its exact position, in both methods. A seeded robot computes its map over absolute cells, and a hop-based beacon marks the cell of the robot that sent it. This drawback was accepted for two reasons. First, real-time kinematic satellite positioning (RTK-GNSS) reaches an accuracy in the order of centimeters, which is far smaller than one grid cell ($10\,\mathrm{m}$ by $10\,\mathrm{m}$) (Grub, 2026). Second, the open sky above a crop field is favorable for satellite positioning. The remaining risk of positioning errors is assessed in Section 9.1.

5
Implementation

This chapter describes how the selected solutions in Chapter 4 were implemented. Section 5.1 first gives an overview of the robot brain, and then describes its parts in turn: the memory, the behaviors, the arbiter, role rebalancing, the potential map and navigation, and the communication between robots. The section closes with the simulation in which the algorithm was developed and evaluated.

5.1 Final Result

The final result of the project was a decentralized swarm algorithm for monitoring and treating pests in a crop field, together with a simulation in which the algorithm was developed and evaluated. Every robot in the swarm ran the same algorithm on its own, using only what it had sensed itself and what nearby peers had told it. No robot had an overview of the whole field, and no robot gave orders to another. Coordinated behavior of the swarm, such as spreading out over the field and dividing the pests among the ground robots, emerged from these local decisions.

Two navigation methods were implemented for the killers, the ground robots that treated the pests. The hop-based killers followed Concept 2, killers walked down beacons that the scout relayed, without a map of the field. The seeded killers followed Concept 3: they planned a route to a known pest on a map built from the pests that the swarm shared. The rest of the brain, the behaviors, the arbiter and the communication, was the same for both, and both swarms used the same scouts, so that the two methods could be compared in Section 7.1.

5.1.1 Overview of the Solution

The solution followed the structure that all three concepts shared in Chapter 3. Each robot answered two questions at every decision: what to do, and which way to move. The first question was answered by the behaviors and the arbiter, and the second by the potential map and the navigator. Between these sat the memory of the robot, which collected everything the robot knew, and the communication with peers, which kept that memory up to date. Figure 5.1 shows how these parts worked together.

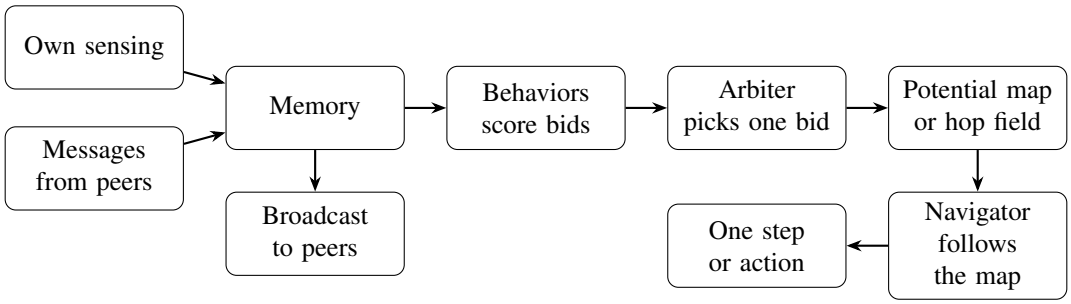


Figure 5.1: The decision cycle of one robot.

A robot went through the following cycle at every decision:

1. it merged what it sensed and what peers had sent it into its memory;
2. every behavior scored how much every possible behavior mattered at that moment;
3. the arbiter chose one behavior, and decided whether it was worth abandoning the current one;
4. the potential map of the chosen behavior was computed, and the robot took one step downhill on it;

5. the robot shared its status and its knowledge with the peers within radio range.

The swarm was heterogeneous: Drones explored the field from the air and could fly over obstacles, ground robots drove between the crop rows and treated the pests, and support robots brought charged batteries to robots that ran low. All robots ran the same decision cycle of Figure 5.1 and differed only in their configuration: which behaviors they carried, how they navigated, how they moved, what they could sense, and which actions their body allowed. A capability check made sure that a robot never chose an action that its body could not carry out, so that, for example, a drone could never decide to treat a pest.

5.1.2 Memory

The memory of a robot held its beliefs about the field. For every cell, it kept the newest observation of the terrain, the crop, and the pest severity, together with the time at which the cell was last observed. For every peer, it kept the newest reported position, battery level, and target. Because information could come from a peer that had observed a cell long ago, older information was trusted less: the confidence in an observed pest decreased with its age.

The memory was the only input to the decisions of a robot. This made the robot independent of a continuous connection with others. Once a peer had shared the location of a pest, the robot could act on it, even if that peer had moved far away or had failed in the meantime. This property was the reason for carrying Concept 3 forward in Section 4.3, because it removed the need for a chain of robots that stayed in contact.

5.1.3 Behaviors

Each activity of a robot was a behavior. A behavior consisted of two parts with a different responsibility. The first part decided *whether* and *how much* the robot should pursue the activity, by producing a score between 0 and 1 from the memory and the state of the robot. The second part decided *where* the activity led, by placing targets with a value on the potential map. Separating the two kept the decision cheap: every behavior could score itself without computing a map, and only the map of the winning behavior had to be computed.

The final solution contained the following behaviors:

Treating. A ground robot that knew of pests scored treating higher as the number of known pests grew relative to the number of known ground robots, so that a robot treated with more urgency when pests outnumbered the robots that could treat them. Its targets were the known pests, each with a value that grew with the severity of the pest and the confidence in the observation. A pest that a peer was already heading for received a lower value, so that the ground robots spread over the pests instead of all heading for the nearest one. A robot standing on a pest scored treating just below the override level, so that only an urgent need, such as an empty battery, could interrupt a treatment.

Exploring. Exploring a new field and monitoring a known one were treated as a single activity: a cell was worth visiting again depending on how long ago anyone had observed it, and a cell that was never observed was simply the oldest information there was. A cell was not worth revisiting until a revisit age A had passed since it was last observed. From then on, its value grew with the age of its information as

$$v(c) = \begin{cases} 0 & \text{if } a(c) < A, \\ 1 - 2^{-a(c)/A} & \text{otherwise,} \end{cases} \tag{5.1}$$

where $v(c)$ was the value of revisiting cell c, between 0 and 1, and $a(c)$ the age of the robot's information about c: the time since c was last observed, by the robot itself or by a peer whose observation had reached

it. A cell that had never been observed had an infinite age. The revisit age A was half the time the scouts needed to visit every free cell once if they divided the field evenly, so that a swarm of any size on any field always had stale cells to spread over. A stale cell was therefore worth 0.5 at the revisit age, and approached 1 for a cell that was never observed. To keep the robots from moving as a herd, the field was divided into blocks of 8×8 cells, and the value of each block was divided by one plus the number of exploring peers that were in it or heading for it.

Recharging and battery support. A robot whose battery ran low returned to a charging station, and a support robot could bring a charged battery to a robot in the field. A battery that was about to run out scored above the override level, so that the robot never stranded itself in the field.

Regrouping. A robot that had lost contact with the rest of the swarm could move back towards its peers, so that information could flow again.

New behaviors could be added without changing the existing ones, because all behaviors scored on the same scale and all used the same potential map for navigation. This met the requirement of one mechanism for every behavior from Chapter 3.

5.1.4 Arbiter

The arbiter followed the design of Section 3.2. It held the robot to a commitment, a pair of an objective and a target, kept a new objective for at least $30\,\mathrm{s}$, and let a rival bid take over only when it beat the commitment by the margin of Equation 3.1. The final solution kept the initial values of the parameters: a base margin of 0.125 between objectives and 0.0625 between targets of the same objective, and an effort weight of 0.375 over a horizon of $5\,\mathrm{min}$.

The final arbiter added one rule before these: a bid that scored 0.9 or higher overrode everything else, without any comparison. This band was reserved for needs that must not be traded off against a good target, such as a battery about to run out, the treatment of the pest the robot stood on, or a switch of role (Section 5.1.5). When several bids reached this band, the highest score won, so a battery about to run out, at 1, came before a treatment, at 0.99, which came before a switch of role, at 0.95.

5.1.5 Role Rebalancing

With fixed roles, the split between scouts and killers was chosen before the first pest appeared: too many killers left the field unwatched, and too few let the pests pile up. Role rebalancing let the swarm change this split during a run. A robot could switch between scout and killer on a dock, which stood for a robot whose role was a module, such as a sensor head or a sprayer, swapped at the home base. Every robot carried the behaviors of both roles, and only the behaviors of its current role could bid.

Every robot made the decision on its own, from its own memory, without a central controller. A robot counted the scouts and the killers it had heard from in the last $20\,\mathrm{min}$, from the status messages that every robot already broadcast, and the pests it knew of. The swarm needed

$$k = \min\left(k_0 + \left\lceil \frac{n_p}{c} \right\rceil,\ \max\left(k_0,\ N - e_{\min}\right)\right) \tag{5.2}$$

killers, where n_p was the number of known pests, $c = 2$ the number of pests one killer was expected to handle, $k_0 = 1$ the killers kept when no pest was known, N the number of scouts and killers the robot had heard from in the last $20\,\mathrm{min}$, and $e_{\min}$ the scouts each robot tried to keep, at least half of the scouts the swarm started with. Because every robot counted from its own memory, the swarm could briefly have fewer scouts than that: with hop-based killers it once had only 3.

Since every robot saw roughly the same shortage, a rule such as “killers are short, so switch” would have made every scout switch at once. Instead, every robot ranked the robots that could switch by their distance to the nearest dock, with ties going to the lowest robot number, and switched only if it was among the first as many robots as were missing. Because all robots used the same rule on nearly the same information, they agreed on who switched without negotiating. A robot on its way to a dock already reported that it was switching, so the other robots counted it in its new role and nobody else set off. On the dock, the switch took $60\,\mathrm{s}$ and refilled the robot’s doses.

Rebalancing used the same idea as the hysteresis of the arbiter, at the level of the swarm instead of a single robot. A shortage or a surplus had to last $2\,\mathrm{min}$ before any robot acted on it, because news arrived in bursts when robots met, and a robot kept its new role for at least $15\,\mathrm{min}$. The bids to drive to a dock and to switch scored 0.95, so they won as an override over the current activity of the robot, except the treatment of the pest it stood on, which scored 0.99.

5.1.6 Navigator and Potential Fields

The role of the navigator was to choose which neighboring cell the robot should move towards at each brain’s decision cycle. To make this choice, the navigator would receive a guidance: a common interface representing either a seeded potential field or a heading generated by the beacons of the hop field. The killer robots used one of the two configurations, while the scouts used solely the seeded-based field. Both types of fields, seed and hop-based, were produced using factory type components, which generated the fields from the robot’s memory and an objective selected by the arbiter.

Seeded-based potential field. The seeded-based field was implementation of the hierarchical quadtree cost propagation of Concept 3 (Equation 3.4), with the distance measured along the routes that each kind of robot could travel. The seeded killers placed the known pests on this map as targets, so a killer drove to a pest along a route that it planned on its own map. The final implementation added one more optimization to the concept apart from the efficient computation using a quadtree. Mainly, a map was only recomputed when something that it depended on had changed, such as a newly discovered obstacle, a treated pest, or the robot entering a new block.

Hop-based potential field. The hop-based killers followed the beacons of Concept 2 (Equation 3.2 and Equation 3.3) and kept no map of the field. A hop-based killer remembered only the newest beacon from each sender, and at most a fixed number of beacons of each kind, so its memory stayed small however large the field or the swarm. Every robot of the swarm, the scouts included, relayed the beacons. The docks broadcast beacons of their own, which a hop-based killer followed in the same way to recharge and restock; since a dock never moved, its beacons never expired. Lastly, if a killer got stuck circling around or standing for a period of time, an escape beacon with repelling force would be generated by the robot such that it would get pushed away from the trap.

5.1.7 Communication

Robots communicated only with peers within radio range. At every decision, a robot broadcast two kinds of messages: its status, consisting of its position, battery level, and current target, and its knowledge, consisting of the observations in its memory. A robot that received a message merged it into its own memory, keeping the newest information per cell and per peer. Information therefore traveled through the swarm by encounters: a drone that found a pest passed it on to every robot it met, and these robots passed it on in turn, until it reached a ground robot that could treat it.

The status messages were what made coordination possible without a central controller. Because a robot knew which targets its peers were heading for, it could lower the value of those targets in its own

potential map, which spread the ground robots over the pests and the drones over the field.

5.1.8 Simulation Environment

The algorithm was developed and evaluated in a simulation of a crop field, which represented the field as a grid in which one cell covered a patch of field rather than a single plant. The simulation modeled obstacles such as rivers and rocks, pests that appeared, grew in severity, and spread to neighboring cells, the movement, sensing, energy use, and treatment of the three kinds of robots, and the limited radio range between them. It could also inject faults: robots that failed during operation, and robots that were wrong about their own position. Every run was reproducible from its random seed, and faults were drawn independently of the pests, so that a run with faults and a run without them faced exactly the same pests.

To make the behavior of the swarm understandable for the Robotics Institute, the simulation came with an interactive viewer. In the viewer, a run could be paused, slowed down, or sped up, a pest could be added by clicking on the field, and a robot could be selected to show what it was doing, which target it was heading for, and what it believed about the field. Runs could be recorded, replayed, and exported as a video, and series of experiments could be described in a configuration file and run without programming.

6
Benchmark Design

The swarm algorithm was evaluated in a simulation, in a set of experiments that each answered one question about the swarm. This chapter describes how these experiments were set up. Section 6.1 introduces the two ways of treating pests that the experiments compared, and Section 6.2 the fields, robots and pests that all experiments shared. Section 6.3 defines the measures by which the runs were compared, Section 6.4 explains how the runs were repeated over random seeds, and Section 6.5 lists the experiments and the question that each one answered. The experiments themselves and their results follow in Section 7.1.

6.1 Compared Methods

Every swarm consisted of scouts, drones that searched the field from the air, and killers, ground robots that drove to the pests and treated them. The scouts were the same in every experiment: they explored the field on the potential map of Concept 3, the role that Chapter 4 assigned to that concept. The experiments compared two ways in which the killers found the pests:

- Hop-based uses the navigation of Concept 2. A robot that saw a pest broadcast a beacon with value 0, and every other robot, the scouts included, broadcast the lowest value it had heard plus one hop. A killer walked down these values to the pest, without a map of the field and without sharing the pests themselves.
- Seeded killers uses the potential map of Concept 3 for the pests as well. The swarm shared the pests it had found, and each killer planned a route over its own map to a known pest. A killer reported the pest it was heading for, and a killer that was farther away left that pest to it.

Both methods had parameters, such as how long a beacon stayed valid or how long a claim on a pest counted. Each was tuned one parameter at a time on the default fields, and all comparisons used the tuned configuration of both methods, so that neither method was compared in a weaker form than it could take.

Some experiments also included the grid search as a baseline without communication: every killer swept its own strip of the field and treated the pests it drove over.

6.2 Default Settings

An experiment used the default settings below, and changed only what its question was about.

Fields. Four standard fields were used, of two sizes, with and without obstacles, as listed in Table 6.1 and shown in Figure 6.1. A cell was $10\,\mathrm{m}$ wide and $10\,\mathrm{m}$ high. The ground robots could cross the rivers only over bridges, while the drones flew over them.

Robots. The default benchmark swarm consists of 10 scouts and 4 killers. The simulation uses three body profiles: SCOUT, KILLER, and BATTERY. These profiles define the movement, sensing, and action capabilities available to each robot; they are modelling assumptions rather than verified physical-hardware specifications. The radio range is 100 m, equivalent to 10 cells of 10 meters. All body types have a terrain-sensing radius and a peer-sensing radius of one cell. SCOUT and KILLER robots can detect pests in their current cell, while BATTERY robots provide battery support and do not detect or treat

Field	Size	Obstacles	Run length
Small, with rivers	32×32 cells	one river with one bridge, and rocks	$2\,\mathrm{h}$
Large, with rivers	64×64 cells	two rivers with three bridges, and rocks	$4\,\mathrm{h}$
Small, open	32×32 cells	none	$2\,\mathrm{h}$
Large, open	64×64 cells	none	$4\,\mathrm{h}$

Table 6.1: The four standard fields.

pests. BATTERY robots are included only in scenarios configured to use battery support, not in the default 10-SCOUT/4-KILLER benchmark swarm. Table 6.2 separates each body's modelled specifications from the actions it can perform.

Body type	Modeled specifications	Capabilities	Constraints
SCOUT	Aerial; maximum speed $8\,\mathrm{m\,s^{-1}}$;	Explores the field, detects pests in its current cell, and can fly over modeled ground obstacles	Cannot treat pests; can receive a replacement battery
KILLER	Ground; maximum speed $1\,\mathrm{m\,s^{-1}}$;	Detects pests in its current cell and treats them	Cannot fly over or cross obstacle cells; can receive a replacement battery

Table 6.2: Robot body profiles implemented in the simulation. Speeds and endurance values are initial model parameters, not guaranteed physical performance.

Pests. Pests were static: they did not grow or spread. Initial pests were placed on the field at the start, and new pests spawned in random free cells throughout the simulation. Each pest started with a severity of 0.1, and a killer continuously treating it cleared it in approximately 6 simulated seconds under the default benchmark settings.

An experiment that deviated from these defaults stated its own settings in its section.

6.3 Performance Measures

Every run was measured with the same set of measures, so that the experiments could be compared with each other. The pest response was measured by:

- the infested cells, the number of infested cells averaged over the run. This was the main outcome: it counted both how many pests stayed untreated and for how long, and it had no ceiling at the end of the run;
- the response time, the minutes from the appearance of a pest to its treatment, averaged over the treated pests, and its 90th percentile. Pests that were still untreated at the end were counted separately, as open pests, so that a swarm could not obtain a low response time by treating only the easy pests;
- the cells per treated pest, the cells driven by all killers together per treated pest, a measure of the effort, and so of the energy, spent per pest;
- the messages, the messages received per robot per hour, a measure of the communication.

The exploration was measured by the coverage, the share of the field that some robot had sensed, the idleness, the minutes since a cell was last sensed averaged over the field, and the time to explore the whole field. The time to clear was the time until a field that was infested at the start was free of pests.

Where an experiment varied the number of robots, the scalability S of (Milner, Sooriyabandara, &

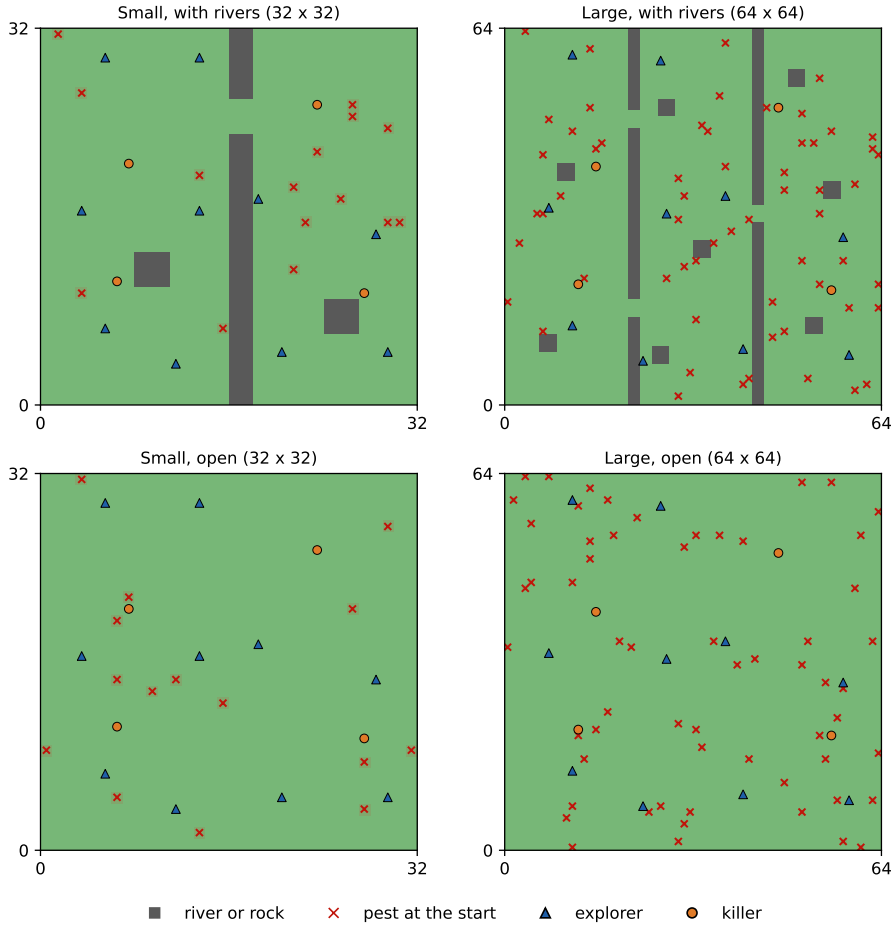


Figure 6.1: The four standard fields at the start of a run with the default swarm, for one random seed. The gaps in the rivers were the bridges; the grey squares were rocks.

Hauert, 2023) compared a swarm of N robots with one of $N + m$ robots, as given in Equation 6.1.

$$S = \frac{(P_{N+m} - P_N)/P_N}{m/N} \tag{6.1}$$

Here P was a score for which higher was better: the inverse of the time to clear the field. A swarm was scalable when $S > 0$, scaled linearly at $S = 1$, and superlinearly above it.

6.4 Running the Experiments

Configurations were run with five random seeds if possible, and the results were reported as the mean over the seeds with its standard error. The seed fixed where and when the pests appeared, where the robots started, and which robots failed when, so two configurations run with the same seed faced exactly the same pests and the same failures. Before each experiment was run, its expected outcome was written down as a set of hypotheses, which helped find errors in the simulation and made clear what each result meant.

6.5 Experiments

Table 6.3 lists the experiments and the question that each one answered. Section 7.1 describes them and their results.

Experiment	Question
BASELINE	How many fewer cells did the seeded and the hop-based swarm leave infested than the grid search, which did not communicate, and how did rivers change that?
EXPLORE	How long did a swarm of scouts take to explore the whole field, following its exploring behavior or sweeping the field in strips?
NUM_EXPLORERS	How many scouts did 5 seeded or 5 hop-based killers need on the small field with a river, at a low and a high pest rate, and how did a radio range of $50\,\mathrm{m}$ instead of $100\,\mathrm{m}$ change that?
SPEED	How long did the seeded killers, the hop-based killers and the grid search take to clear a field that was infested at the start, and how did that change with the number of killers?
RADIO	How did a radio range of $50\,\mathrm{m}$, $100\,\mathrm{m}$ and $300\,\mathrm{m}$ affect the seeded and the hop-based killers?
SCALE	What was the smallest swarm that cleared $90\,\%$ of the pests within $15\,\mathrm{min}$, and how did it grow with the size of the field?
RESUPPLY	How many more robots did a swarm need when it had to recharge and restock at a dock, and did a second dock help?
OUTAGE	How much worse did the seeded and the hop-based killers perform when robots failed during the run, and did it matter whether a failed robot went dark or kept relaying messages?
REBALANCE	Did letting the robots switch roles during the run leave fewer cells infested than fixed roles, with seeded and with hop-based killers?

Table 6.3: The experiments and the question that each one answered.

7
Validation

This chapter validates the swarm algorithm of Chapter 5. Section 7.1 evaluates the swarm as a whole in the experiments of Chapter 6, each of which answered one question about the swarm. Section 7.2 then discusses the results as a whole: the patterns that held across the experiments, how the seeded and the hop-based killers met the requirements of Chapter 3, and the trade-offs between them. Finally, Section 7.3 discusses the limitations of the validation. Throughout the project, the correctness of the implementation was additionally safeguarded by a suite of automated tests, which checked every change before it was accepted.

7.1 Experiments

Each experiment answered one of the questions of Table 6.3. The experiments compared the hop-based killers with the seeded killers, and some also compared them with the grid search, a baseline without communication. Every experiment used the default settings of Chapter 6, and its subsection states only where it deviated from them. The expected outcomes were written down before the runs, so each subsection first lists them and then discusses the results against them. The experiments build on each other. The baseline experiment first shows whether a swarm is worth its communication at all. The next two experiments look at the scouts: how fast they explore a field, and how many of them the killers need. The clearing speed and the radio range experiments then compare how the two kinds of killers respond to pests. The last four experiments cover what a real deployment adds: larger fields, resupply at a dock, robot failures, and robots that switch roles during a run.

7.1.1 Baseline (BASELINE)

This experiment compared the two swarms with the grid search, the baseline without communication. It used two large fields of 64×64 cells that differed only in their terrain. The open field had no obstacles. The river field had two rivers with three bridges, and eight rocks. Each swarm had 5 scouts and 5 killers, and the grid search had 5 killers without a radio. Both fields started with 8 static pests and received 8 more every hour for $4\,\mathrm{h}$. The infested cells were averaged from the end of the first hour, after the pests present at the start had been cleared. Every system was run with 12 seeds.

The grid search was a fair baseline: a robot that does not communicate and senses a pest only on its own cell can at best patrol its own part of the field in a fixed cycle (Chevaleyre, 2004). The grid search reached this bound on the open field, with $68\,\mathrm{min}$ on average since a cell was last sensed, and came within $15\,\%$ of it on the river field.

Two outcomes were expected. First, both swarms would keep far fewer cells infested than the grid search. Second, the rivers would hurt the hop-based killers but not the seeded killers. A seeded killer plans its route around a river on its map. A hop-based killer follows beacons, and a beacon crosses a river over the radio, where the killer cannot follow.

Both swarms beat the grid search. As expected, both swarms kept far fewer cells infested than the grid search on both fields, the seeded swarm about a fifth as many (Table 7.1). The grid search swept whether or not there was a pest, so a new pest waited on average half a sweep, about an hour, and the infested cells stayed high for the whole run (Figure 7.1).

Field	System	Infested cells	Response (min)	p90 (min)	Cells per pest
Open	seeded	**2.17 ± 0.13**	15.2 ± 0.4	25.9 ± 0.6	28 ± 1
	hop-based	3.82 ± 0.20	26.5 ± 0.9	55.9 ± 3.5	88 ± 2
	grid search	10.10 ± 0.57	61.0 ± 2.2	119.1 ± 2.9	223 ± 10
Rivers	seeded	**2.20 ± 0.10**	14.9 ± 0.4	25.9 ± 0.7	30 ± 1
	hop-based	**6.81 ± 0.77**	37.9 ± 2.8	91.3 ± 9.6	113 ± 4
	grid search	11.13 ± 0.49	67.1 ± 2.4	141.9 ± 4.1	229 ± 12

Table 7.1: The seeded and the hop-based swarm against the grid search on the open and the river field: the infested cells averaged after the first hour, the minutes to treat a pest and its 90th percentile, and the cells driven by the killers per treated pest. Mean ± standard error over 12 seeds. In green, the seeded swarm, which the rivers did not affect; in red, the hop-based swarm on the river field, which the rivers hurt most. Fields of 64×64 cells, open or with two rivers and eight rocks; 8 static pests at the start and per hour, 4 h. Swarms of 5 scouts at $8\,\mathrm{m\,s^{-1}}$ and 5 killers at $1\,\mathrm{m\,s^{-1}}$ with a 100 m radio; grid search of 5 killers without a radio.

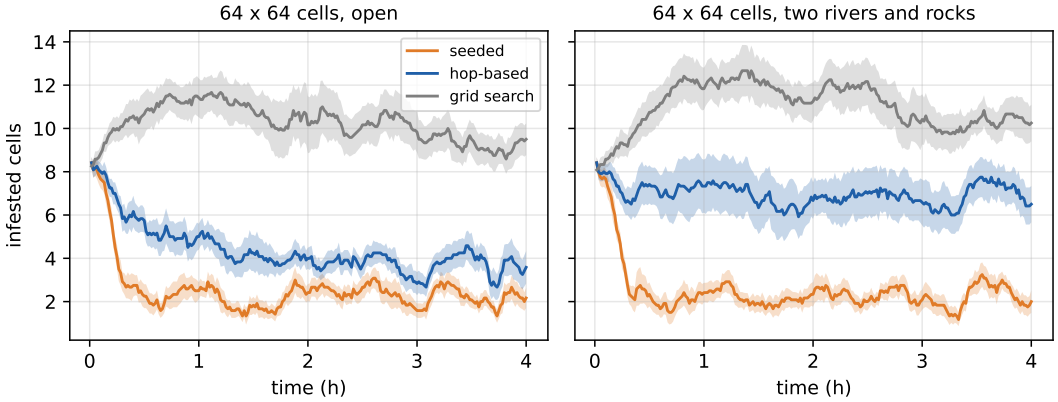


Figure 7.1: The infested cells over the run on the open (left) and the river field (right), for the seeded and the hop-based swarm and the grid search; mean with a band of one standard error over 12 seeds. Settings as in Table 7.1.

The rivers hurt the hop-based swarm, but not the seeded one. As expected, the rivers left the seeded swarm unchanged, while the hop-based swarm went from 3.8 to 6.8 infested cells (red in Table 7.1). It also became less predictable: it brought the infested cells down to two in 11 of the 12 runs on the open field, but in only 6 on the river field. A hop-based killer followed a beacon to the bank of a river, and then had to find a bridge without a map.

Even without rivers, the hop-based swarm lagged behind. On the open field, the hop-based swarm still held 1.65 ± 0.16 more infested cells than the seeded swarm. With 10 robots on 4 096 cells, the swarm was too sparse for its beacons to reach the killers reliably (Section 7.2). Of the gap of 4.6 cells on the river field, about 3 cells were thus due to the rivers and the rest to the sparse swarm.

7.1.2 Exploration (EXPLORE)

This experiment measured how long a swarm of 1, 5, 10 or 20 scouts took until every cell of the field had been sensed at least once, when the scouts followed their exploring behavior, and when they swept the field in fixed strips instead, as the grid search did. It used the small and the large field with rivers, with scouts only: no killers and no pests. A run stopped as soon as the whole field was explored. Besides the time to explore the whole field, the times at which half and nine tenths of it were explored, t_{50} and t_{90}, were measured.

Before the runs, four outcomes were expected:

1. every configuration would explore the whole field, except perhaps one scout on the large field;
2. the time to explore would fall about as $1/n$ with the number of scouts n, since each scout would cover its share of the cells at the same rate;
3. the grid search would explore the whole field faster, since a sweep visited every cell exactly once, while the exploring scouts headed for the stalest cells and crossed each other's paths;
4. t_{90} would be closer between the two methods than the time to explore the whole field, since the exploring scouts would cover most of the field quickly and spend their last minutes on scattered cells they had missed.

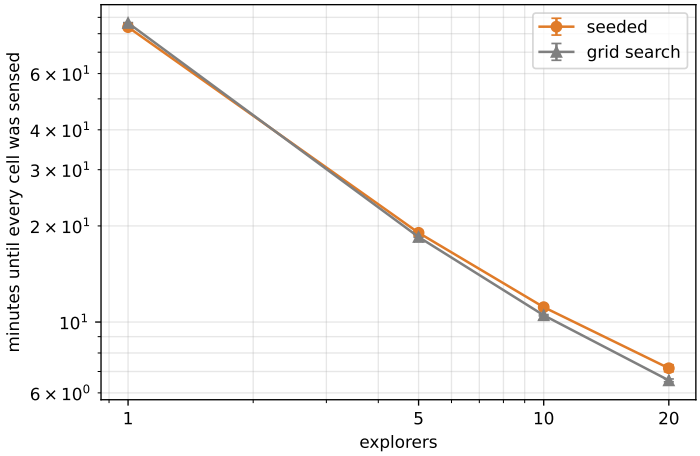


Figure 7.2: Minutes until a swarm of scouts had sensed every crop cell of the large field with rivers (64×64 cells, two rivers and rocks), against the number of scouts, for the exploring behavior and the grid search; mean $\pm$ standard error over 5 seeds. Radio range $100\,\mathrm{m}$, scouts at $8\,\mathrm{m\,s^{-1}}$, no killers.

Figure 7.2 shows the time to explore the large field against the number of scouts, and Table A.1 in Appendix A lists the results on both fields. Every configuration explored the whole field, including a single scout on the large field, which took $84\,\mathrm{min}$. The second expectation was only partly confirmed: from 1 to 20 scouts the time to explore the large field fell by a factor of 12 for the exploring scouts and 13 for the grid search, against 20 for perfect scaling. Both methods fell short because a run lasted until the last cell was sensed, so the slowest scout set the time, and because each scout first had to move to its own part of the field, which took as long with 20 scouts as with one. The third expectation was not confirmed: neither method was consistently faster, and on the large field the two were within $10\,\%$ of each other in every configuration. The exploring scouts did cover half of the field sooner than the grid search in every configuration on the large field, with a t_{50} of $2.3\,\mathrm{min}$ against $2.9\,\mathrm{min}$ for 20 scouts, and t_{90} was closer between the methods than the time to explore the whole field, as expected. The exploring behavior therefore lost nothing against a fixed sweep in the first exploration of a field, while it also kept revisiting the field afterwards, which a single sweep did not.

7.1.3 Number of Scouts (NUM_EXPLORERS)

This experiment looked at how many scouts the killers needed. The scouts searched the field and passed on the pest information they found, and the killers drove to those pests and treated them. The experiment used the small field with a river and 5 killers, either seeded or hop-based, with 1, 2, 5, 10 or 20 scouts. Two pest rates were used, a low rate of 20 new pests per hour (4 per killer) and a high rate of 60 new pests per hour (12 per killer). At the start of each run, the field already had as many pests as arrived in one hour. Every setup was run with a radio range of $100\,\mathrm{m}$ and of $50\,\mathrm{m}$, for $2\,\mathrm{h}$, 5 times with different random start point.

Before the runs, three outcomes were expected:

1. with a 100 m radio, the seeded killers would hardly improve beyond about 5 scouts, one per killer;
2. the hop-based killers would gain more from extra scouts than the seeded killers, because their messages needed a chain of robots to reach a killer, but they would still be worse than the seeded;
3. at the low pest rate, the number of scouts would hardly matter, because the killers would have time to spare.

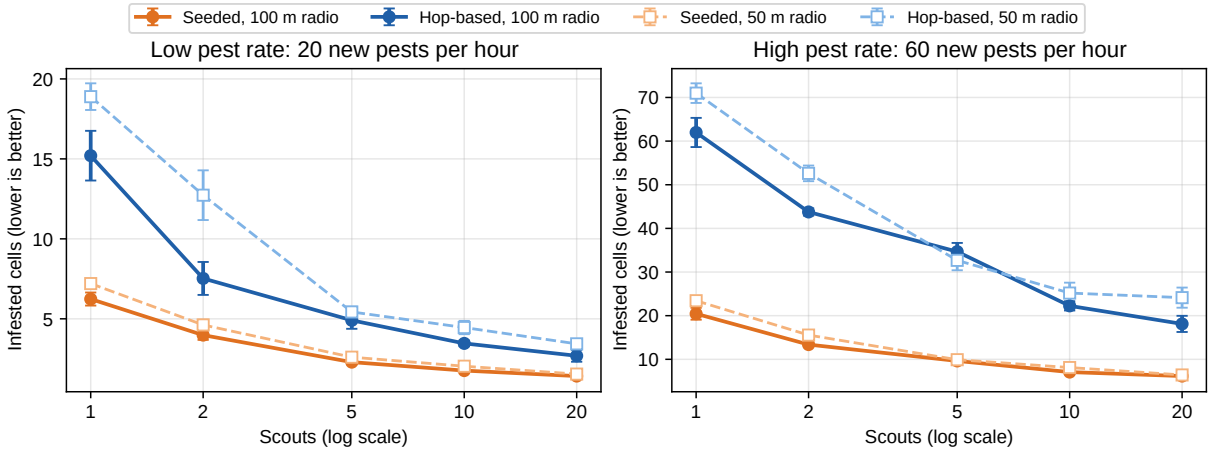


Figure 7.3: Infested cells averaged over the runs against the number of scouts (lower is better), at the low (left) and the high (right) pest rate, for 5 hop-based (blue) or 5 seeded (orange) killers. Small field with a river (32 × 32 cells), no relays, scouts at 8 m s^{-1}, killers at 1 m s^{-1}, 20 or 60 static pests at the start and 20 or 60 per hour, 2 h, averaged from the start of the run.

		Seeded killers		Hop-based killers	
Pest rate	Scouts	100 m	50 m	100 m	50 m
Low	1	6.2 ± 0.4	7.2 ± 0.2	15.2 ± 1.6	**18.9 ± 0.8**
	5	2.3 ± 0.1	2.6 ± 0.2	4.9 ± 0.5	5.4 ± 0.3
	20	**1.4 ± 0.1**	1.5 ± 0.05	2.7 ± 0.4	3.4 ± 0.2
High	1	20.4 ± 1.4	23.4 ± 0.9	62.0 ± 3.3	**71.0 ± 2.2**
	5	9.6 ± 0.9	9.9 ± 0.4	34.7 ± 2.0	32.7 ± 2.3
	20	**6.2 ± 0.2**	6.4 ± 0.3	18.1 ± 1.9	24.1 ± 2.3

Table 7.2: Average number of infested cells during the run (lower is better) with 1, 5 and 20 scouts, for 5 seeded or 5 hop-based killers with a radio range of 100 m or 50 m. In green, the fewest infested cells at each pest rate; in red, the most. Mean ± standard error over 5 seeds; small field with a river, 2 h per run; the low pest rate was 20 and the high pest rate 60 new pests per hour. Table A.2 lists every number of scouts.

Figure 7.3 and Table 7.2 show the infested cells; Appendix A lists every number of scouts in Table A.2 and the response times in Table A.3.

More scouts kept helping, also at the low pest rate. In every setup, the infested cells fell as scouts were added, although each doubling helped less than the one before (Figure 7.3). The seeded killers still improved from 10 to 20 scouts, so the first expectation was wrong: one scout per killer was not enough. The third expectation was wrong as well. At the low pest rate, the scouts cut the infested cells by at least as large a share as at the high rate, because with killers to spare, finding the pests was what took time.

Seeded killers were better, and a short radio hurt them less. At every number of scouts, the hop-based killers left two to four times as many cells infested as the seeded killers. They gained more cells from extra scouts but did not catch up, so the second expectation was only partly right. A 50 m radio changed little for the seeded killers, because the fast scouts carried the news of a pest to the killers

themselves. It hurt the hop-based killers in most setups, because their beacons needed robots close enough to pass them on (the dashed blue lines in Figure 7.3).

How many scouts are enough? The answer depends on how fast a pest must be treated. For a target of $10\,\mathrm{min}$ on average from the appearance of a pest to its treatment, 2 scouts were enough for 5 seeded killers in every setup except the high rate with a $50\,\mathrm{m}$ radio, which needed 5 (Table A.3). The hop-based killers needed 10 scouts at the low rate, and did not reach the target at the high rate even with 20.

Limitations. The infested cells were averaged from the start of the run, so at the high rate they partly measured how fast the swarm cleared the pests present at the start. The parameters of the hop-based killers were tuned with 10 scouts and may have suited other numbers of scouts less well. The response time counted only the treated pests, which flattered the setups that left many pests untreated.

7.1.4 Clearing Speed (SPEED)

This experiment measured how long 1, 5, 10, 20 and 40 killers took to clear the large field when $1\,\%$ of it was infested at the start, with seeded killers, hop-based killers, or killers that swept the field in strips, as in the grid search. It used the large field with rivers, with 38 pests at the start, $1\,\%$ of its crop cells, and no new pests afterwards. The seeded and the hop-based killers worked with the default 10 scouts, which found the pests for them; the grid search consisted of the killers alone, each sweeping its own strip and treating what it drove over. A run stopped as soon as the field was clear, or after $12\,\mathrm{h}$.

Before the runs, four outcomes were expected:

1. the time to clear would fall with the number of killers for every method, about as $1/n$ for the grid search, whose strips shrank with the number of killers, and less for the swarm;
2. with few killers, the seeded and the hop-based killers would clear the field far sooner than the grid search, since the scouts found all pests within minutes, and the killers drove only to the pests;
3. with many killers, the grid search would catch up, and might overtake the swarm at 40 killers;
4. the seeded killers would clear the field sooner than the hop-based killers.

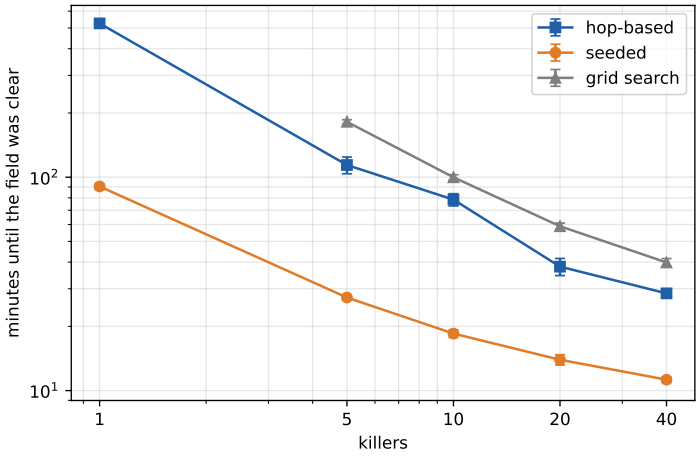


Figure 7.4: Minutes until the field was clear of the pests that infested it at the start, against the number of killers, for the hop-based killers, the seeded killers and the grid search; mean ± standard error over the seeds that cleared the field within $12\,\mathrm{h}$, out of 5. Large field with rivers (64×64 cells), 38 static pests ($1\,\%$ of the crop cells) at the start and none after; the hop-based and the seeded killers with 10 scouts at $8\,\mathrm{m\,s^{-1}}$, the grid search without; killers at $1\,\mathrm{m\,s^{-1}}$, radio $100\,\mathrm{m}$.

	Time to clear (min)			Response (min)		
Killers	Seeded	Hop-based	Grid search	Seeded	Hop-based	Grid search
1	**90.5 ± 2.8**	–	–	43.2 ± 2.0	229.4 ± 15.4	355.6 ± 10.1
5	**27.3 ± 0.6**	114.1 ± 10.3	181.9 ± 4.0	11.9 ± 0.6	42.2 ± 2.6	92.6 ± 4.6
10	**18.5 ± 0.8**	78.5 ± 5.2	99.9 ± 2.8	8.2 ± 0.3	24.0 ± 0.6	47.0 ± 1.6
20	**14.0 ± 0.8**	38.1 ± 3.5	58.9 ± 2.1	6.4 ± 0.2	14.1 ± 0.9	27.6 ± 0.9
40	**11.3 ± 0.3**	28.6 ± 1.3	39.9 ± 1.7	5.6 ± 0.3	8.8 ± 0.8	19.1 ± 0.6

Table 7.3: Minutes until the field was clear, and the mean minutes to treat a pest (response); mean ± standard error over 5 seeds. A dash means that some seed did not clear the field within 12 h: a single hop-based killer treated 88 % and a single sweeper 82 % of the pests, while every other configuration treated all of them. In green, the seeded killers, which cleared the field fastest at every number of killers. Large field with rivers (64×64 cells), 38 static pests (1 % of the crop cells) at the start and none after; the hop-based and the seeded killers with 10 scouts at $8\,\mathrm{m\,s^{-1}}$, the grid search without; killers at $1\,\mathrm{m\,s^{-1}}$, radio 100 m.

Figure 7.4 shows the time to clear the field against the number of killers, and Table 7.3 lists it together with the response time. The time to clear fell with the number of killers for every method, as the first expectation stated.

The seeded killers cleared the field fastest at every number of killers. With 5 killers they needed 27 min, against 114 min for the hop-based killers and 182 min for the grid search, which confirmed the second and the fourth expectation. The grid search stayed the slowest method even with 40 killers, so the third expectation was not confirmed.

The seeded killers were limited by finding the pests. They gained less from each doubling of the killers: their scalability (Equation 6.1) fell from 0.48 to 0.24. With 40 killers they cleared the field in 11 min, about the time that 10 scouts needed to explore the large field (Section 7.1.2).

The hop-based killers needed enough killers to relay their beacons. From 10 to 20 killers they scaled superlinearly, with a scalability of 1.06, likely because the killers then became dense enough to pass each other's beacons on. With 40 killers they cleared the field in 29 min, sooner than the grid search.

7.1.5 Radio Range (RADIO)

This experiment measured how the radio range affected the seeded and the hop-based killers. It used the default swarm on the small field with a river and on the small open field, with a radio range of 50 m, 100 m and 300 m for every robot: five, ten and thirty cells.

Before the runs, four outcomes were expected:

1. both methods would get worse as the radio got shorter, because news of a pest would take longer to reach a killer;
2. the hop-based killers would suffer more from a short radio than the seeded killers, since a beacon reached a killer only along a chain of robots within range of each other, while a seeded killer still learned of a pest later, through the shared map;
3. at 300 m nearly every robot would hear every other, and the two methods would come close;
4. the river would make both methods slower, but would not change how they ranked.

Figure 7.5 shows the infested cells and the response time against the radio range on the field with a river, and Table 7.4 lists the results on both fields.

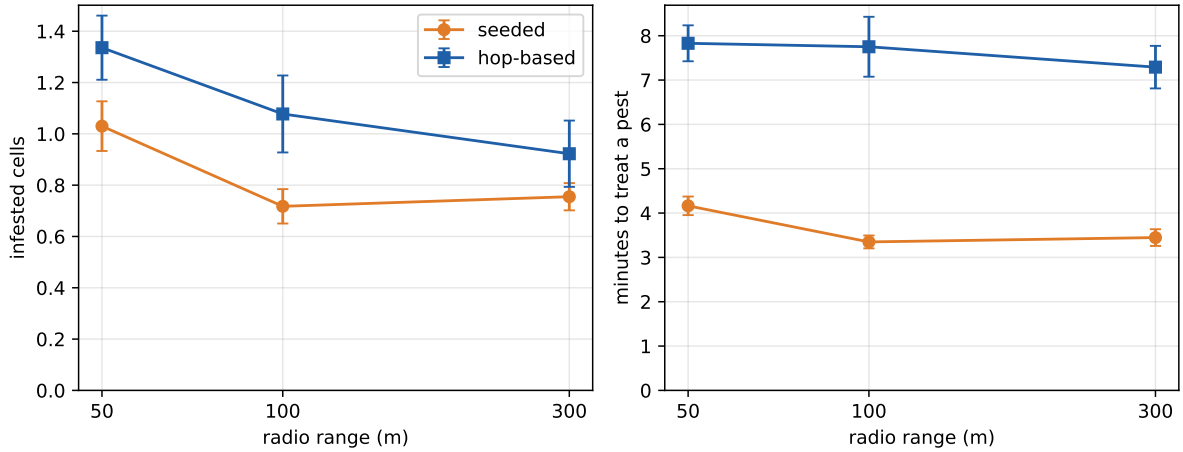


Figure 7.5: Infested cells averaged after the first hour (left), and the mean minutes from the appearance of a pest to its treatment (right), against the radio range, for the seeded and the hop-based killers on the small field with a river (32×32 cells, one river and rocks); mean $\pm$ standard error over 5 seeds. 10 scouts at $8\,\mathrm{m\,s^{-1}}$ and 4 killers at $1\,\mathrm{m\,s^{-1}}$; 15 static pests at the start and 15 per hour, over $2\,\mathrm{h}$.

Only the shortest radio hurt, and a longer radio cost communication. At $50\,\mathrm{m}$ both methods left more cells infested than at $100\,\mathrm{m}$ (red in Table 7.4), but a longer radio brought no further gain, so the first expectation held only for the shortest radio. From $50\,\mathrm{m}$ to $300\,\mathrm{m}$ the messages grew about twentyfold for both methods. The second expectation was not confirmed: the hop-based killers did not consistently suffer more from the short radio than the seeded killers.

The seeded killers were better at every range. They treated a pest in 3 to $4\,\mathrm{min}$, against 6 to $8\,\mathrm{min}$ for the hop-based killers, and drove four to five times fewer cells per treated pest, because they planned a route to a pest instead of following the beacons. At $300\,\mathrm{m}$ the two methods left about as many cells infested, as the third expectation stated, but the hop-based killers still needed twice as long to treat a pest. The river hurt the hop-based killers at $100\,\mathrm{m}$ and the seeded killers at $50\,\mathrm{m}$, but the methods ranked the same on both fields, as the fourth expectation stated.

With 15 pests per hour, 4 killers kept up with the pests in every configuration, so the infested cells stayed around one; a higher pest rate would separate the configurations more clearly.

7.1.6 Scalability (SCALE)

The clearing speed experiment (Section 7.1.4) showed that on a field of fixed size, every extra killer helped less than the one before. A farmer with a larger field asks a different question: how many robots does the field need to keep its pests under control? This experiment answered that question for fields of four sizes. On each field, it searched for the smallest swarm that met a required response, and it measured how that swarm grew with the field.

Setup. The fields were open, without obstacles, so that only their size changed. Their sides were 32, 45, 64 and 90 cells, so each field had about twice the area of the one before, from 1 024 to 8 100 cells, or 10 to $81\,\mathrm{ha}$. The pests had the same density on every field: 10 static pests appeared per hour per 1 024 cells, and as many were present at the start. Every swarm had 2.5 scouts per killer, rounded up. All other settings were the defaults of Chapter 6, including a radio range of $100\,\mathrm{m}$. A run lasted $4\,\mathrm{h}$, and its first hour was left out. On each field, the number of killers was varied around the swarm that just met the requirement, from 1 to 4 killers on the smallest field up to 6 to 24 on the largest. Every configuration was run with 3 seeds instead of 5, because a run of the largest swarm took about $3\,\mathrm{h}$ of computing.

Field	Killers	Radio	Infested cells	Response (min)	Cells per pest	Messages
River	seeded	50 m	**1.03 ± 0.10**	4.16 ± 0.21	14.1	2 295
		100 m	0.72 ± 0.07	3.35 ± 0.15	8.9	9 836
		300 m	0.76 ± 0.05	3.45 ± 0.19	9.3	48 166
	hop-based	50 m	**1.34 ± 0.13**	7.83 ± 0.41	43.5	3 335
		100 m	1.08 ± 0.15	7.75 ± 0.68	48.0	14 668
		300 m	0.92 ± 0.13	7.29 ± 0.48	46.7	72 384
Open	seeded	50 m	**0.82 ± 0.10**	3.95 ± 0.14	12.5	2 296
		100 m	0.72 ± 0.12	3.61 ± 0.06	9.5	9 833
		300 m	0.71 ± 0.12	3.49 ± 0.02	8.3	48 143
	hop-based	50 m	**1.25 ± 0.22**	7.69 ± 0.33	40.0	3 454
		100 m	0.73 ± 0.08	5.64 ± 0.23	43.5	15 431
		300 m	0.78 ± 0.16	7.45 ± 1.14	45.1	72 536

Table 7.4: The seeded and the hop-based killers at three radio ranges, on the small field with a river and on the small open field: the infested cells averaged over the run, the minutes to treat a pest, the cells driven by the killers per treated pest, and the messages received per robot per hour. In red, the shortest radio, with which both methods left the most cells infested on both fields. Mean ± standard error over 5 seeds; 10 scouts at $8\,\mathrm{m\,s^{-1}}$ and 4 killers at $1\,\mathrm{m\,s^{-1}}$, 15 static pests at the start and per hour, 2 h.

Measures. On every field, the experiment looked for the smallest swarm that cleared at least 90 % of the pests within 15 min of their appearance. Only the pests that appeared after the first hour and at least 15 min before the end of the run were counted, and a pest that was never cleared counted as late. The swept swarms had whole numbers of killers, so the smallest swarm usually lay between two of them. On the smallest field, for example, one seeded killer cleared 88 % of the pests in time and two cleared 100 %. Drawing a straight line between these two results, it crosses 90 % at 1.2 killers. This number of killers is $k_{\min}$, and $N_{\min}$ is the number of robots of that swarm, killers and scouts together. How the smallest swarm grew with the number of cells C of the field was described by the exponent α of a power law,

$$N_{\min} \propto C^{\alpha}, \tag{7.1}$$

fitted by least squares on logarithmic axes. With $\alpha = 1$, doubling the field doubles the robots needed. With α below 1, a field twice as large needs less than twice the robots, and with α above 1 more. The scalability S of Equation 6.1 asks how much a fixed task gains from extra robots, while α asks how many robots a growing task needs for the same service. The required 15 min kept the smallest swarm within the swept swarms on every field.

Expectations. Linear scaling, $\alpha = 1$, would be perfect scalability: twice the robots serve a field twice as large. A swarm that scales worse needs more than twice the robots for twice the field, with α above 1. Both methods were hoped to scale linearly, because their robots act only on what they and their neighbours know, and no robot coordinates the whole field. The hop-based killers were expected to need a larger swarm on every field. They have no map: they find a pest by following beacon values that their peers pass on, and these values point the way only where the peers are spread densely enough over the field.

The seeded killers improved steadily, the hop-based killers only above a threshold. Figure 7.6 shows how the share of pests cleared in time grew with the density of the swarm. The seeded swarm gained from its first robots on and met the requirement from about 4.3 robots per 1 024 cells. The hop-based swarm followed an S-shaped curve instead: below about 4 robots per 1 024 cells it cleared only 10 to 30 % of the pests in time, and it met the requirement only from about 8.5 robots. This threshold lay where the robots were spaced about one radio range apart, a square of about 110 m per robot against

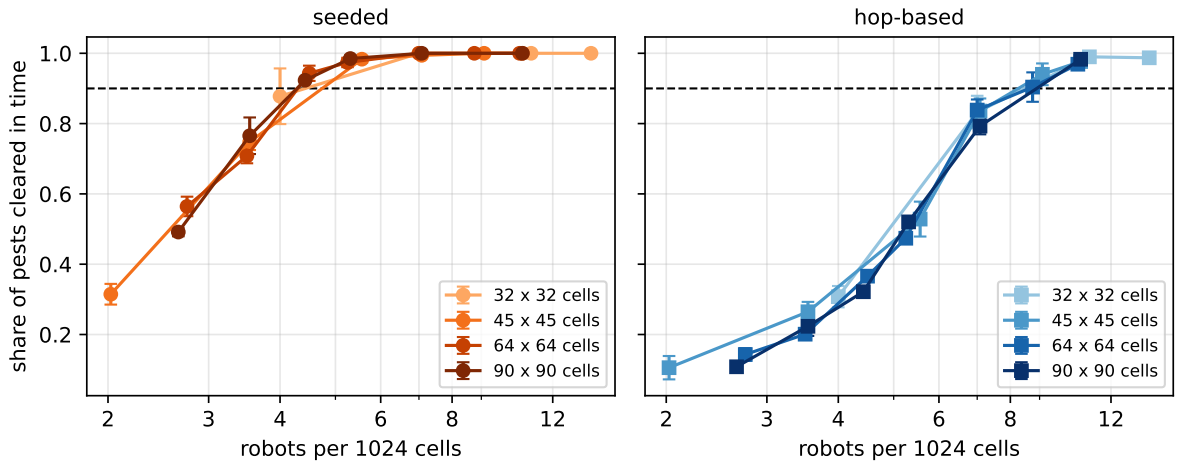


Figure 7.6: The share of the pests cleared within 15 min against the robots per 1 024 cells of the field, for the seeded (left) and the hop-based killers (right), with a line per field; mean ± standard error over 3 seeds, and the requirement of 0.9 dashed. Open fields of 1 024 to 8 100 cells with 10 static pests per hour per 1 024 cells and as many at the start; 2.5 scouts per killer, rounded up; 4 h runs with the first hour left out.

		Seeded			Hop-based		
Field	Cells	$k_{\min}$	$N_{\min}$	per 1 024 cells	$k_{\min}$	$N_{\min}$	per 1 024 cells
32×32	1 024	1.2	4	**4.1**	2.4	8	**8.5**
45×45	2 025	2.6	9	**4.7**	4.6	16	**8.2**
64×64	4 096	4.8	17	**4.2**	9.9	35	**8.7**
90×90	8 100	9.7	34	**4.3**	20.5	72	**9.1**
α			**1.00**			**1.04**	

Table 7.5: The smallest swarm that cleared 90 % of the pests within 15 min, per field: its killers $k_{\min}$, its robots $N_{\min}$, killers and scouts together, and its robots per 1 024 cells; and the exponent α of Equation 7.1. In green, the robot density, which stayed about the same on every field, and the exponents, both close to 1. Settings as in Figure 7.6.

a radio range of 100 m. Below it, the beacon values had gaps that the killers could not follow, so a hop-based swarm that was too small failed almost completely, while a seeded swarm that was too small lost performance gradually.

Both methods scaled linearly, and the hop-based killers needed twice the robots. For both methods, the four fields gave nearly the same curve, and doubling the field doubled the smallest swarm (Figure 7.7 and Table 7.5), with $\alpha = 1.00$ for the seeded and 1.04 for the hop-based killers. This also held with a stricter requirement of 10 min. Unlike in the clearing speed experiment, the scouts grew with the killers and the field with both, so every robot kept a patch of the same size. On every field, the hop-based killers needed 1.8 to 2.1 times the robots of the seeded killers, as expected. For the business case, a larger farm thus needs proportionally more robots: with about one new pest per hectare per hour, a seeded swarm needed one killer and two to three scouts for every 8.4 ha.

Limitations. With 3 seeds, the smallest swarms are uncertain by up to about one killer. The fields were open, and since rivers almost doubled the infested cells of the hop-based swarm in the baseline experiment (Section 7.1.1), the hop-based killers would likely need more than twice the robots on fields with rivers. Finally, every swarm had 2.5 scouts per killer, so the results do not show whether a larger field needs more scouts or more killers.

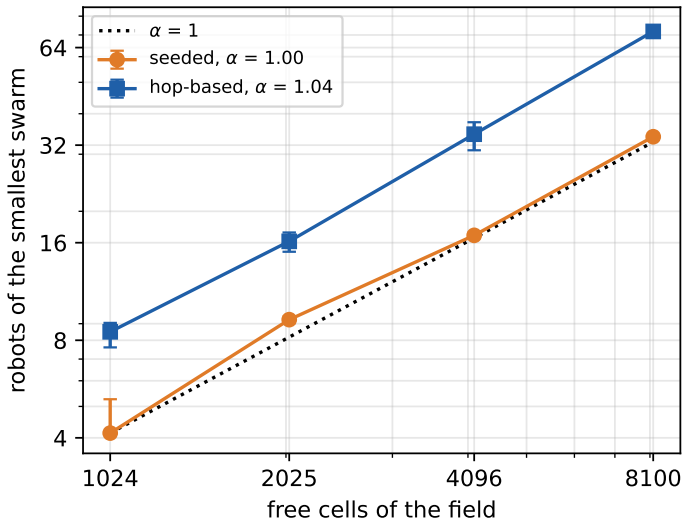


Figure 7.7: The robots of the smallest swarm against the cells of the field, on logarithmic axes, with the fitted exponent α and the line of linear scaling, $\alpha = 1$, dotted. The error bars show where the mean share plus and minus one standard error reached 0.9. Settings as in Figure 7.6.

7.1.7 Resupply (RESUPPLY)

In the other experiments, the robots never ran out of battery or doses. Real robots have to leave the field to recharge and restock at a dock, and while they do so they neither search nor treat. This experiment asked how many more robots a swarm needs when it has to resupply, and how many docks a field needs: one in a corner, or a second one on the other side of the field, which on the river field is the other bank.

Setup. The experiment used a field of 32×32 cells, either open or with the river, its bridge near the top, and two rocks. Each field had three layouts. In the unlimited layout, the robots never ran out, as in the scalability experiment. The other two layouts had a single dock in the bottom left corner, or a second dock in the bottom right corner, on the other bank of the river. With docks, a scout's battery lasted $15\,\mathrm{min}$ of flight and a killer's $60\,\mathrm{min}$ of driving, and a killer carried 5 doses. These values were not meant to be realistic. They were chosen so that both kinds of robots left the field regularly. A robot headed for a dock when a quarter of its battery was left, and the dock swapped its battery in $5\,\mathrm{min}$ and refilled its doses. A hop-based killer found a dock the same way as a pest: by following the beacons that the dock sent out and the robots passed on, without a map. The pests, the swarms and the runs were those of the scalability experiment (Section 7.1.6): 10 static pests per hour and as many at the start, 2.5 scouts per killer, rounded up, and $4\,\mathrm{h}$ runs with the first hour left out. The number of killers was varied from 1 to 6, and up to 12 for the hop-based killers with one dock on the river field. Every configuration was run with 8 seeds.

Measures. As in the scalability experiment, the smallest swarm k_{min} was the number of killers with which the swarm cleared $90\,\%$ of the pests within $15\,\mathrm{min}$ of their appearance, interpolated between the swept swarms. The cost of resupply was the difference between the smallest swarm with docks and the unlimited one.

Expectations. Resupply was expected to cost every swarm robots, since a robot on its way to a dock does not work. On the open field, a second dock was expected to help little, because a corner of a small field is never far away. On the river field, one dock was expected to cost more, because the killers on the far bank have to drive round by the bridge. This was expected to hurt the hop-based killers most. A

seeded killer plans its route to the dock on its map. A hop-based killer follows the dock's beacons, which cross the river over the radio, where the killer cannot follow.

Field	Killers	Unlimited	Two docks	One dock
Open	seeded	1.3 (5)	1.9 (7)	2.0 (7)
Open	hop-based	2.2 (8)	3.5 (12)	3.3 (12)
River	seeded	1.4 (5)	1.9 (7)	2.5 (9)
River	hop-based	2.9 (10)	3.9 (14)	> 12 (> 42)

Table 7.6: The smallest swarm that cleared 90 % of the pests within 15 min: its killers $k_{\min}$, interpolated between the swept swarms, and in brackets its robots, killers and scouts together. In red, the hop-based killers with one dock on the river field, where no swarm swept met the requirement. Fields of 32 × 32 cells, open or with a river; 10 static pests per hour and as many at the start; 2.5 scouts per killer, rounded up. With docks, a scout's battery lasted 15 min of flight, a killer's 60 min of driving, and a killer carried 5 doses. 4 h runs with the first hour left out; 8 seeds.

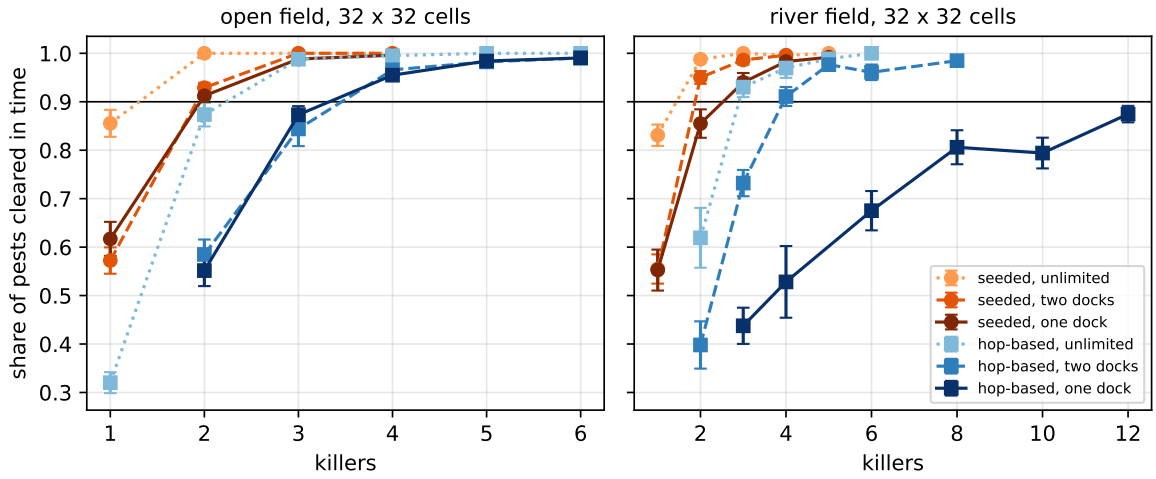


Figure 7.8: The share of the pests cleared within 15 min against the killers of the swarm, on the open (left) and the river field (right), for the seeded (orange) and the hop-based killers (blue), without resupply (dotted), with two docks (dashed) and with one dock (solid); mean ± standard error over 8 seeds, and the requirement of 0.9 as a line. Settings as in Table 7.6.

Resupply cost about half as many killers again. As expected, every swarm needed more robots with docks (Table 7.6): on the open field, about 50 % more killers for both methods. The scouts spent about 30 % of their time on their way to a dock or on it, and the killers 6 to 32 %, less as the swarm grew (Figure 7.9).

A second dock mattered only across the river. On the open field, one dock in a corner was enough: the longer trips cost a killer only a few minutes an hour, and the smallest swarm stayed the same. On the river field, one dock cost the seeded killers more, 2.5 killers instead of 1.9, because a killer on the far bank drove round by the bridge, but no seeded robot ran out of battery.

With one dock on the river field, the hop-based killers failed. No swarm swept met the requirement, up to 12 killers and 42 robots (Figure 7.8), and 1.5 to 5 robots per run stopped with an empty battery. In seven traced runs, all 21 killers that ran out had set off for the dock from the far bank. Thirteen never crossed the river, most of them stopping right on its bank, where the dock's beacons had led them, and

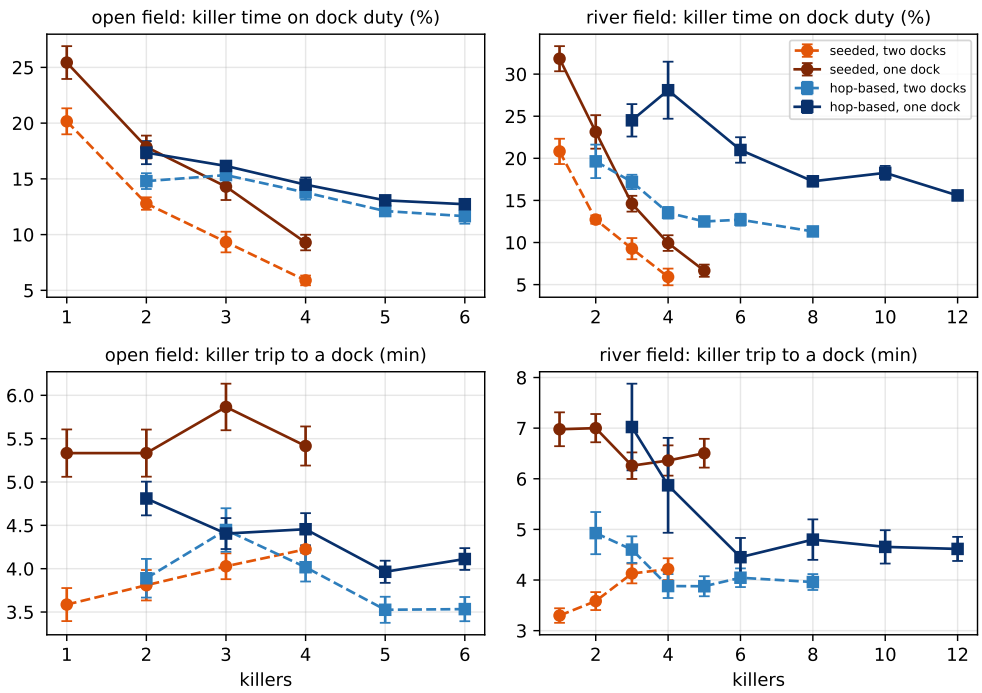


Figure 7.9: The share of a killer's time on its way to a dock or on it (top), and the duration of a completed trip to a dock (bottom), against the killers of the swarm, on the open (left) and the river field (right), with two docks (dashed) and with one dock (solid). Trips that ended with an empty battery are left out. Mean ± standard error over 8 seeds, and over the trips for their duration. Settings as in Table 7.6.

the other eight found the bridge too late. This confirms the last expectation: where the radio crosses an obstacle that a killer cannot, the beacons lead a hop-based killer astray, so a field with hop-based killers needs a dock on every part that the killers cannot leave without a detour.

Limitations. The battery and the doses were chosen to make resupply frequent, not to match real robots, and the dock swapped a battery in 5 min, while charging would keep a robot away much longer. The experiment used only the small field; on a larger field, a second dock would likely matter more, also on open ground.

7.1.8 Robot Failures (OUTAGE)

This experiment measured how much worse the seeded and the hop-based killers performed when robots failed during the run, and whether it mattered that a failed robot went dark or kept communicating. It used the default swarm on the small field with a river, for 2 h. Every minute, every working robot, scout or killer, failed with a probability of 0.0042, so that on average 40 % of the swarm had failed by the end. A failed robot either went dark, as if it was removed from the field, or became immobile: it stopped moving and treating, but still sensed its own cell and passed on what it heard, which corresponded to a motor failure. The failures were drawn independently of the pests, so with the same seed every configuration with failures lost the same robots at the same moments and faced the same pests.

Before the runs, five outcomes were expected:

1. both methods would get worse as robots failed, so the infested cells would rise during a run with failures, while they stayed level without;
2. the hop-based killers would degrade more than the seeded killers, since their beacons traveled only along chains of working robots, while a seeded killer kept the pests it already knew of;

3. the difference between the methods would grow with the robots lost, so it would be largest at the end of the run;
4. immobile robots would hurt the hop-based killers less than dark ones, since the chains along which the beacons traveled kept their links;
5. for the seeded killers, immobile robots would hurt about as much as dark ones, or more, since an immobile killer could hold on to a pest near it that it could never reach, while a farther killer left that pest to it.

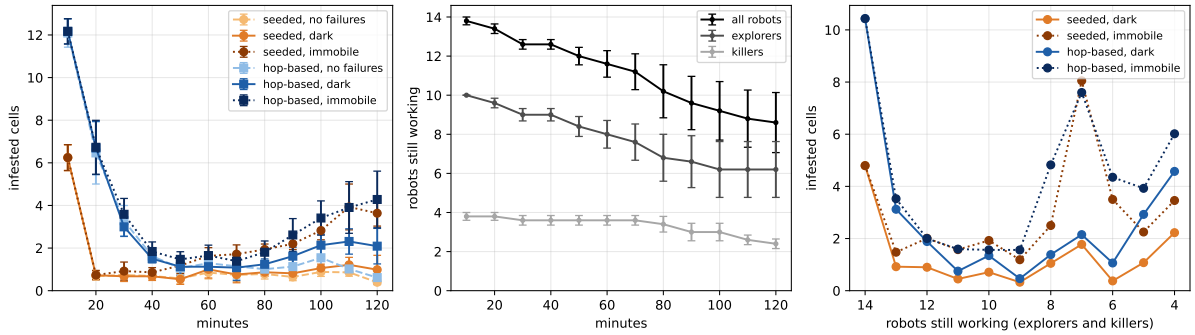


Figure 7.10: Robot failures, for the seeded (orange) and the hop-based killers (blue). Left: the infested cells over time without failures (light, dashed), with failed robots that went dark (solid), and with failed robots that stayed in place and kept communicating (immobile, darkest, dotted), averaged over each $10\,\mathrm{min}$. Middle: the robots still working over time, all robots and the scouts and the killers among them; the failures were drawn alike for the same seed, so this held for every configuration with failures. Right: the infested cells against the robots still working. Every run with failures was cut into periods of $10\,\mathrm{min}$, and each point is the mean of the infested cells over all periods, of every seed, that ended with that many robots working. A run started with all 14 robots and lost them over time, so the axis runs forward in time; the first point includes the first minutes, while the pests present from the start were still being found, and the last points rest on few periods at the end of a few runs. Mean $\pm$ standard error over 5 seeds, left and middle. Small field with a river (32×32 cells), $2\,\mathrm{h}$; 10 scouts at $8\,\mathrm{m\,s^{-1}}$ and 4 killers at $1\,\mathrm{m\,s^{-1}}$, radio $100\,\mathrm{m}$; 15 static pests at the start and per hour. Every robot failed with a probability of 0.0042 per minute, $40\,\%$ of the swarm by the end on average.

Killers	Failures	Infested cells	Response (min)	Open pests	Idleness (min)
seeded	none	0.72 ± 0.07	3.35 ± 0.15	0.8	1.3
seeded	dark	0.95 ± 0.13	3.57 ± 0.19	1.8	2.6
seeded	immobile	2.71 ± 0.26	4.08 ± 0.34	4.6	7.7
hop-based	none	1.08 ± 0.15	7.75 ± 0.68	1.2	1.2
hop-based	dark	1.75 ± 0.33	8.32 ± 0.74	2.8	2.4
hop-based	immobile	2.90 ± 0.76	9.27 ± 1.03	5.4	7.1

Table 7.7: The seeded and the hop-based killers without failures, and with $40\,\%$ of the swarm failing during the run, either going dark or becoming immobile: the infested cells averaged over the run, the minutes to treat a pest, the pests still infested at the end, and the minutes since a cell was last sensed. Mean $\pm$ standard error over 5 seeds; small field with a river, 10 scouts and 4 killers, radio $100\,\mathrm{m}$, 15 static pests at the start and per hour, $2\,\mathrm{h}$.

Figure 7.10 shows the infested cells and the robots still working over the run, and Table 7.7 summarizes the runs. Both methods got worse with failures, as the first expectation stated.

Robots that went dark hurt the hop-based killers more. The infested cells rose by $32\,\%$ for the seeded and by $62\,\%$ for the hop-based killers, as the second expectation stated: a seeded killer kept the pests it already knew of, while the beacons lost the robots that carried them. As the third expectation stated, the gap grew as robots failed: once fewer than about ten robots were left, the hop-based killers rose to about two infested cells while the seeded killers stayed near one (Figure 7.10). The large difference at the very start of the run came from finding the pests present from the beginning, not from the failures.

Immobile robots hurt both methods most. For the seeded killers, immobile robots almost quadrupled the infested cells, from 0.7 to 2.7, as the fifth expectation stated: an immobile killer still reported the pest it was heading for, so the other killers left that pest to it, although it would never arrive. For the hop-based killers, immobile robots were worse than dark ones too, against the fourth expectation, although the difference was within about one and a half standard errors, and towards the end of the run both methods rose together. Immobile scouts also slowed the exploration, with the idleness rising from about $1\,\mathrm{min}$ to more than $7\,\mathrm{min}$ (Table 7.7), likely because they still reported that they were exploring, so the other scouts kept away from their part of the field.

7.1.9 Role Rebalancing (REBALANCE)

With fixed roles, the split between scouts and killers was chosen before the first pest appeared. Rebalancing, described in Section 5.1.5, let the swarm change that split during the run: when there had been too few killers or too few scouts for $2\,\mathrm{min}$, the robots closest to a dock drove there and switched role.

This experiment compared fixed roles with rebalancing of roles on the small field (32×32 cells) with a river, with the default swarm split of 10 scouts and 4 killers, either seeded or hop-based, with their tuned parameters. In the rebalancing swarm every robot started with the same role and at the same place as in the fixed swarm. The field started with on average of 20 static pests and on average 20 more appeared every hour, the radio range was $100\,\mathrm{m}$, and every setup was run for $2\,\mathrm{h}$, 5 times with different random seeds. With the same seed, both setups started with the same pests.

Before the runs, three outcomes were expected for rebalancing:

1. at the start, the swarm would turn scouts into killers, and give some of them back as the backlog was cleared;
2. with seeded killers, rebalancing would leave fewer cells infested than fixed roles, because the extra killers would work through the backlog and the scouts would come back once it was gone;
3. with hop-based killers, rebalancing would help less than with seeded killers, or even hurt.

Killer type	Roles	Infested	Δ infested	Response	Slowest 10 %	Open	Avg. killers	Switches
Seeded	Fixed	2.53 ± 0.20	–	4.89 ± 0.30	9.5 ± 0.5	3.4 ± 0.8	4	0
Seeded	Rebalancing	2.59 ± 0.06	$+0.07 \pm 0.20$	5.38 ± 0.26	8.8 ± 0.8	2.2 ± 0.2	**2.7 (1–8)**	17.6
Hop-based	Fixed	5.54 ± 0.65	–	11.16 ± 0.81	27.6 ± 1.5	2.8 ± 0.5	4	0
Hop-based	Rebalancing	5.52 ± 0.38	-0.03 ± 0.28	9.64 ± 0.60	**20.3 ± 1.9**	**4.4 ± 0.5**	3.9 (2–11)	20.8

Table 7.8: Fixed roles against rebalancing, for seeded and hop-based killers: the infested cells averaged over the run, the difference in infested cells to fixed roles with the same run (Δ infested), the minutes to treat a pest (response) and the 90th percentile of those minutes (slowest 10 %), the pests still infested at the end (open), the killers averaged over the run, with the fewest and the most at any moment in brackets, and the role switches per run. Lower is better for every column except the killers, where fewer means more robots exploring. In green, the gains of rebalancing: the seeded swarm needed fewer killers for the same result, and the hop-based swarm treated its slowest pests sooner; in red, its cost: the hop-based swarm left more pests open at the end. Mean ± standard error over 5 runs; small field (32×32 cells) with a river, 10 scouts and 4 killers at the start, radio $100\,\mathrm{m}$, on average 20 static pests at the start and 20 per hour, $2\,\mathrm{h}$.

Figure 7.11 and Table 7.8 show the results. In short: rebalancing kept the field as clean as fixed roles with both kinds of killers, and, as in the other experiments, the kind of killer mattered far more than whether the roles were fixed.

The swarm followed the pests. The first expectation held. The rebalancing swarm added killers when many pests were known and gave them back later, with between 1 and 8 seeded or between 2 and 11 hop-based killers during a run (Table 7.8). The waiting time before a switch and the minimum time in a

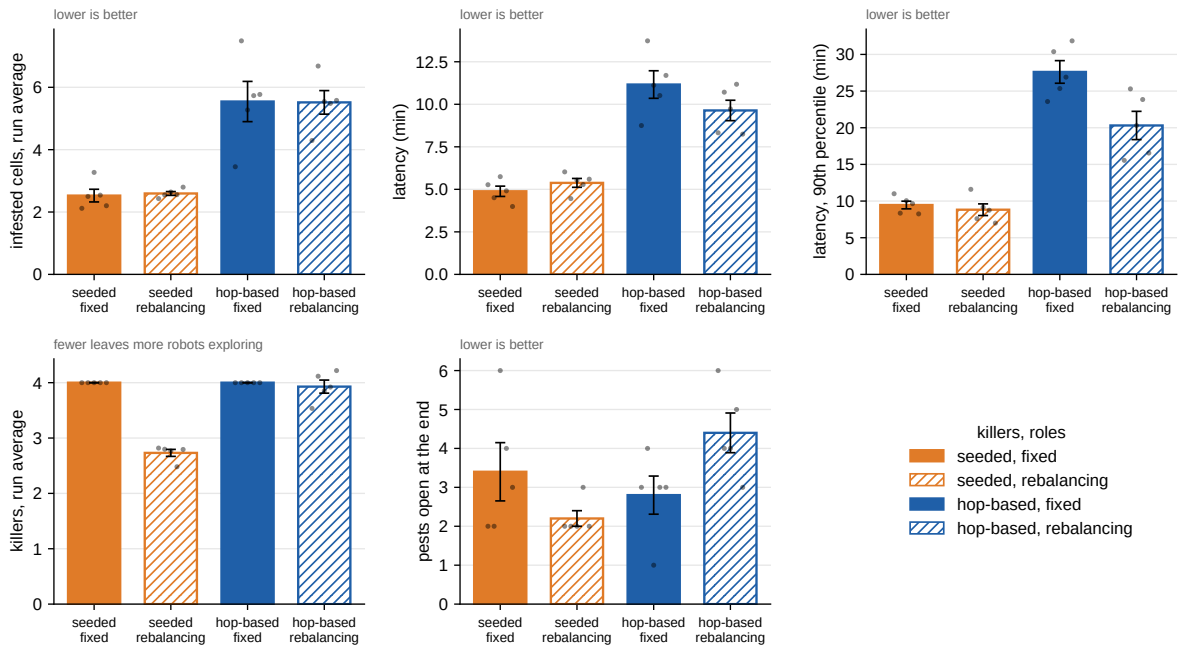


Figure 7.11: Fixed roles against rebalancing, for seeded (orange) and hop-based (blue) killers. The panels show the infested cells averaged over the run, the minutes to treat a pest and its 90th percentile, the killers averaged over the run, and the pests still infested at the end; lower is better in every panel except the killers, where fewer means more robots exploring. The bars show the mean ± standard error over 5 seeds, and the gray dots show the result of each of the 5 runs, one per seed. Small field with a river (32×32 cells), 10 scouts at $8\,\mathrm{m\,s^{-1}}$ and 4 killers at $1\,\mathrm{m\,s^{-1}}$ at the start, radio $100\,\mathrm{m}$, on average 20 static pests at the start and 20 per hour, $2\,\mathrm{h}$; a switch took $60\,\mathrm{s}$ on a dock, and rebalancing kept at least 5 scouts.

role kept the switching low, at fewer than one switch per robot per hour.

Rebalancing kept the field as clean, with fewer killers. The second expectation was not confirmed. With seeded killers, rebalancing left as many cells infested as fixed roles: on this field, 4 fixed killers already kept up with the pests, which left rebalancing little to gain. It did reach this result with 2.7 killers on average instead of 4, so about a third of the killers could explore instead.

With hop-based killers, the slowest pests were treated sooner. The third expectation was not confirmed either. With hop-based killers, rebalancing again left as many cells infested as fixed roles, but the slowest $10\,\%$ of the pests waited about $7\,\mathrm{min}$ less, at the cost of a few more pests left open at the end of the run. Part of this gain may have come from the map that a robot built as a scout and kept when it switched to a hop-based killer.

7.2 Discussion

The experiments of Section 7.1 each answered one question. This section combines them: it first draws out the patterns that held across the experiments, then judges the two navigation methods against the requirements.

7.2.1 Patterns Across the Experiments

The seeded killers were better in every experiment. No setup was found in which the hop-based killers outperformed the seeded killers. The seeded killers treated pests sooner in every experiment. In infested cells, the gap ranged from none, when four killers easily kept up with the pests (Section 7.1.5), to a factor of two to four when the pests were many or the swarm was sparse (Section 7.1.1, Section 7.1.3 and Section 7.1.9). The seeded killers also needed about half the robots for the same service (Section 7.1.6) and cleared a field two to four times sooner (Section 7.1.4). Both swarms beat the grid search, so communication paid off with either method (Section 7.1.1).

The hop-based killers depended on a dense, connected swarm. Most of the differences between the methods trace back to one property. A beacon reaches a killer only through a chain of robots within radio range, and it points along the path of the radio, not along a path that the killer can drive. This explains why the hop-based killers fell behind in sparse swarms and worked well only with about one robot per radio range (Section 7.1.1 and Section 7.1.6), why they scaled superlinearly once the killers relayed each other's beacons (Section 7.1.4), why rivers and a single dock led them astray (Section 7.1.1 and Section 7.1.7), and why they lost more when robots went dark or when the radio was short and the scouts were few (Section 7.1.8 and Section 7.1.3). The seeded killers kept what they had learned in their own memory, so the fast scouts carried the news of a pest across gaps in the swarm, and the map routed the killers around obstacles.

For the seeded killers, finding the pests was the bottleneck. Extra scouts kept helping at every pest rate (Section 7.1.3), while extra killers helped less and less once the time to clear a field approached the time to explore it (Section 7.1.4). Where the killers already kept up with the pests, rebalancing turned a third of them into scouts without losing performance (Section 7.1.9), and a radio longer than $100\,\mathrm{m}$ added communication but no gain (Section 7.1.5). Since the scouts explored a new field as fast as a fixed sweep (Section 7.1.2), and the smallest swarm grew linearly with the field (Section 7.1.6), a seeded swarm can be sized per area: about one killer and two to three scouts for every $8.4\,\mathrm{ha}$ at one new pest per hectare per hour.

Both methods shared two weak spots. Failed robots that still claimed their pests hurt both methods more than robots that disappeared (Section 7.1.8), and resupply at a dock cost both about half as many killers again (Section 7.1.7).

7.2.2 The Methods Against the Requirements

Table 7.9 summarizes how the two methods met the six requirements of Section 3.1, and two further properties that the experiments tested: stable decisions and robustness to failures. The seeded killers met seven of them fully and were only partly robust to failures. The hop-based killers met fewer. A hop-based killer followed the sum of the pulls of its beacons, so it could be caught between two pests that pulled equally hard, and it needed escape beacons to break free (Req. 1). Its beacons crossed obstacles over the radio that the killer could not cross, which made rivers costly and stranded killers on the wrong bank (Req. 2 and Req. 3). A hop-based swarm also used two mechanisms instead of one: the hop field for its killers and the potential map for its scouts (Req. 5).

7.3 Limitations of the Validation

The validation was carried out entirely in simulation, because no physical robots were available within the project. The simulation represented the field as a coarse grid, assumed that every robot knew its

Requirement	Evidence	Seeded	Hop-based
Req. 1: one target	The potential map led every cell to exactly one target (Equation 3.4); a hop-based killer headed for its strongest beacon, and killers pushed each other away (Equation 3.3), but a killer could be caught between two pests that pulled equally hard.	Yes	Partly
Req. 2: obstacles and motion	Seeded killers planned their routes over drivable cells, and rivers did not change their infested cells; the beacons crossed rivers over the radio, where hop-based killers could not follow, and rivers almost doubled their infested cells (Section 7.1.1).	Yes	Partly
Req. 3: never stuck	Every reachable cell had a neighbor with a lower potential, with wall following as a fallback, and no seeded robot ran out of battery on the way to a dock; with one dock on the river field, 21 hop-based killers ran out of battery on the wrong bank (Section 7.1.7).	Yes	Partly
Req. 4: affordable	The quadtree computed a map eight times faster than a map of every cell (Table 3.1); a hop-based killer kept a bounded number of beacons.	Yes	Yes
Req. 5: one mechanism	Treating, exploring, recharging, restocking, regrouping and switching role all bid on the same scale and used the same navigation (Section 5.1).	Yes	No
Req. 6: collective	Robots lowered the value of the targets their peers headed for; both swarms kept far fewer cells infested than the grid search (Section 7.1.1), and the smallest swarm grew linearly with the field (Section 7.1.6).	Yes	Yes
Stable decisions	The arbiter held a commitment for a minimum time and required a margin to switch (Section 3.2); with rebalancing, robots switched role less than once per hour (Section 7.1.9).	Yes	Yes
Robust to failures	With $40\,\%$ of the robots going dark, the infested cells rose by $32\,\%$ for the seeded and by $62\,\%$ for the hop-based killers; immobile robots, which still claimed their pests, almost quadrupled them for the seeded killers (Section 7.1.8).	Partly	Partly

Table 7.9: The final solution against the requirements, for the seeded and the hop-based killers. The first six requirements are those of Section 3.1.

position within one cell, and modeled sensing, treatment, and energy use with simplified rules. The parameters of the arbiter and the potential map were tuned in the tested scenarios, so they may need to be tuned again for other fields, crops, or robots. In addition, the pests were static, detection and communication within radio range were perfect, and the pest rates were often low enough for the killers to keep up, which left little room for differences between configurations. Most configurations were run with only 3 to 5 seeds, so some differences lay within the standard error. Testing the algorithm on hardware, first with hardware in the loop and then in a small field trial, is part of the recommendations in Chapter 11.

8

Business Case and Deployment Potential

This chapter assesses the potential business value of the developed swarm algorithm and the conditions required for its commercialization. The analysis considers the algorithm as a software product rather than a complete agricultural robot. Section 8.1 describes the business opportunity, and Section 8.2 identifies the relevant stakeholders. Section 8.3 presents the proposed commercial model. Section 8.4 describes the relationship between the algorithm and the physical robot platforms. Section 8.5 estimates the software productization cost, while Section 8.6 examines potential revenue and the break-even point. Finally, Section 8.7 considers the potential economic value for agricultural users, and Section 8.8 identifies the remaining requirements for commercialization.

8.1 Business Opportunity

The developed algorithm provides decentralized coordination for agricultural robot swarms. Its main functions include behavior selection, target allocation, and navigation towards tasks such as field exploration, pest treatment, and charging. By allowing robots to make decisions using local information and peer-to-peer communication, the algorithm provides an alternative to coordination approaches that depend on a central controller.

The potential business value lies in providing the coordination software to companies that manufacture agricultural robots or integrate robotic systems. These companies could incorporate the algorithm into their products instead of developing a complete swarm coordination solution internally. The algorithm could therefore become a reusable software component within a broader agricultural robotics product.

Digital agricultural technologies can improve input efficiency and support more targeted crop management, although the economic benefits depend on the technology, crop, and operating conditions (Papadopoulos et al., 2024). The commercial opportunity for the Swarm Field algorithm is to support these benefits through decentralized coordination rather than through the manufacture of physical robots.

Three potential sources of revenue can be identified:

1. **Software licensing:** granting robot manufacturers or system integrators the right to use the algorithm in their products.
2. **Integration services:** adapting the software to specific robot interfaces, communication protocols, and operating requirements.
3. **Maintenance and support:** providing software updates, compatibility improvements, and technical support after deployment.

This business model separates the responsibilities of the software provider from those of the hardware manufacturer. The software provider develops and maintains the coordination algorithm, while the hardware partner supplies the physical platforms and their onboard systems. Commercial deployment would nevertheless require integration with physical robots and validation under real operating conditions.

8.2 Stakeholder Analysis

The commercialization of the algorithm involves stakeholders with different levels of influence and interest. Mendelow's power–interest matrix categorizes stakeholders according to their power to influence

a project and their interest in its outcome (Mendelow, 1981). The matrix provides a basis for determining how each stakeholder group should be engaged.

Table 8.1 presents a preliminary classification of stakeholders relevant to the Swarm Field algorithm. Their positions may change as the project progresses from research toward commercial deployment.

	Low interest	High interest
High power	**Keep satisfied:** Regulatory authorities and standards bodies that can influence deployment requirements, even when they are not directly involved in selecting the software.	**Manage closely:** The Robotics Institute as the research stakeholder, together with agricultural robot manufacturers and system integrators as potential commercial partners. These parties can influence software development, integration, and adoption.
Low power	**Monitor:** Members of the general public outside initial deployment areas and organizations with limited direct involvement in agricultural robotics.	**Keep informed:** Farmers, field workers, and agronomists who may benefit from targeted pest treatment and reduced monitoring effort, but have limited direct influence over software development decisions.

Table 8.1: Power–interest matrix for the commercialization of the Swarm Field algorithm.

Robot manufacturers and system integrators are particularly important because they provide a route from the software to physical deployment. Cooperation with these companies would allow the algorithm to be evaluated against actual hardware constraints, including positioning accuracy, communication range, robot speed, and treatment capability.

The Robotics Institute also has an important role in determining how the research output will be maintained and transferred beyond the project. Software ownership, licensing rights, and responsibilities for maintenance must be established before commercialization.

Farmers and field operators are the downstream users of the technology. Their willingness to adopt the system will depend on whether the complete robotic solution provides measurable benefits compared with existing monitoring and treatment methods. Their feedback should therefore inform future field trials and the assessment of commercial viability.

8.3 Commercial Model

A business-to-business licensing model is a suitable starting point for the Swarm Field algorithm. FY2026 is used as the financial base year, and all financial model figures are expressed in FY2026 euros. The software provider licenses the coordination algorithm to agricultural robotics companies or system integrators, which incorporate it into their platforms and remain responsible for the physical product and deployment.

The license could cover a product, fleet, or integration partner, with pricing based on deployed robots, supported configurations, or operational scale. An annual license would generate recurring revenue and fund software maintenance as requirements evolve.

Integration services could provide additional revenue through interface development, configuration, testing, and platform adaptation. Annual maintenance agreements could cover defect correction, compatible releases, documentation, and technical support.

Licensing can generate revenue from repeated use of the same software, whereas integration requires additional customer-specific engineering. The model should therefore limit custom development and

maximize reuse across platforms.

8.4 Hardware Platform and Integration Scope

The Swarm Field project focuses on the development of the swarm coordination algorithm and its simulation, rather than the design and manufacture of physical robots. The hardware platform, including the sensors, communication equipment, locomotion systems, batteries, and charging infrastructure, is outside the scope of the algorithm team's deliverable. Nevertheless, the cost and capabilities of these components influence the commercial feasibility of the complete system and must therefore be considered in the business analysis.

The proposed software is intended to coordinate heterogeneous robot platforms with different capabilities and operational roles. Aerial robots can support field surveying and communication relaying, while ground robots can perform monitoring and treatment tasks. Dedicated logistics platforms could provide charging support and improve fleet availability. The algorithm determines how these robots select and coordinate their tasks, whereas the physical capabilities of each platform determine which tasks it can perform.

Table 8.2 presents published cost benchmarks for existing agricultural robots and integrated robotic systems. These examples provide a reference for the capital investment associated with agricultural robotics, although the systems differ in their capabilities, equipment, and intended applications.

System	Reported cost	Basis of estimate
Agrobot Lala	Approximately €50,000	Integrated in-field soil-sampling and nitrate-analysis system, including a commercial unmanned ground vehicle estimated at approximately €40,000 (Kitić et al., 2022).
Mechanical weeding robots	€75,000–240,000	Price range reported from interviews concerning six commercially available robots (Shang et al., 2023).
Spot-spraying robots	€80,000–100,000	Price range reported from interviews concerning two commercially available robots (Shang et al., 2023).
Agricultural robot for seeding and weeding	€144,500	Purchase price assumed for the robot with a seeding attachment in an operational and economic assessment (Vahdanjoo et al., 2023).

Table 8.2: Published cost benchmarks for agricultural robots and integrated robotic systems.

The reported figures show that agricultural robotic systems can require substantial investment. However, they cover different tasks, equipment configurations, and estimation methods, and originate from different publication years. As they have not been adjusted for FY2026 inflation, they should be treated as historical benchmarks rather than current procurement prices. An integrated robot's purchase price also does not represent the cost of an individual swarm unit with fewer capabilities.

These benchmarks do not establish the cost of a Swarm Field fleet. Estimating it would require selecting platforms, sensors, treatment equipment, and robot numbers by role, while accounting for assembly, integration, charging, maintenance, and replacement.

The algorithm team develops the coordination software, while a hardware provider or system integrator supplies the physical platforms. The software must interface with positioning, sensors, battery state, treatment capabilities, movement commands, and communication systems. Cooperation between the partners would support adaptation to different platforms.

Reusing the algorithm across hardware configurations is possible, but compatibility cannot be assumed. Differences in positioning, communication range, speed, sensing, and treatment equipment require integration and validation. The software provider must account for productization, licensing, maintenance, and integration, while the hardware partner accounts for the platforms and deployment. Customers must consider the combined system and operating costs.

8.5 Software Development Costs

Commercialization requires the research implementation to be prepared for use by external companies. This includes software packaging, interface development, automated testing, technical documentation, integration testing, and deployment tooling. These costs relate to the algorithm software and do not include manufacturing or purchasing the physical robots.

Knab reports an average hourly rate of €94, excluding value-added tax, for self-employed software developers in the Netherlands in 2026 (Knab, 2026). This rate is used as a reference for estimating engineering expenditure. The required engineering hours and contingency are assumptions for the present business case rather than measured project costs.

Assuming that productization requires between 1,600 and 2,400 engineering hours, the estimated development cost is shown in Table 8.3.

Cost item	Lower estimate	Upper estimate
Engineering effort	1,600 hours	2,400 hours
Engineering rate	€94/hour	€94/hour
Engineering expenditure	€150,400	€225,600
Contingency (20%)	€30,080	€45,120
Total productization cost	**€180,480**	**€270,720**

Table 8.3: Indicative one-time cost of preparing the algorithm for commercial deployment.

The estimated productization cost is therefore approximately €180,000–271,000. The engineering effort represents the time required to prepare the software for commercial use, rather than the cost of developing the algorithm during the research project.

The estimate excludes robot manufacturing, sensors, batteries, charging infrastructure, hardware maintenance, field trials, certification, sales and marketing, legal costs, and general company overhead. These expenses require separate estimates before a commercial investment decision.

Annual software maintenance is assumed to require 15% of the initial productization cost. This results in an estimated maintenance budget of approximately €27,000–41,000 per year in FY2026. The percentage is a modeling assumption and should be refined using actual maintenance requirements after deployment. In subsequent years, maintenance costs are adjusted for inflation as described in Section 8.6.

8.6 Revenue and Break-Even Analysis

The financial feasibility of the software business depends on the number of customers, the achievable license price, and the cost of maintaining the product. Three hypothetical annual license prices are considered: €5,000, €10,000, and €15,000 per integration partner. These values are scenario assumptions rather than observed market prices for the Swarm Field algorithm.

The model applies annual inflation of 2.5% from FY2027 onwards. The European Commission's Spring 2026 forecast projected Dutch inflation of 3.2% in 2026 and 2.5% in 2027 (European Commis-

sion, 2026). As FY2026 is the base year, 2.5% is used for subsequent years as a simplifying assumption.

For the central cost estimate, 2,000 engineering hours at €94 per hour result in €188,000 of engineering expenditure. Including the 20% contingency gives a total productization cost of €225,600. Annual maintenance is estimated at 15% of this amount, or €33,840.

To estimate the annual revenue required to recover the initial investment over three years, the productization cost is divided over the recovery period and added to the annual maintenance cost. The initial productization cost is treated as a fixed FY2026 investment, while recurring maintenance costs increase with inflation:

$$C_{\text{annual},t} = \frac{C_{\text{development}}}{T} + C_{\text{maintenance},0}(1+i)^t, \tag{8.1}$$

where $C_{\text{development}}$ is the initial productization cost, T is the assumed recovery period in years, $C_{\text{maintenance},0}$ is the FY2026 maintenance cost, i is the annual inflation rate, and t is the number of years after FY2026.

For the central scenario, with $T = 3$ years,

$$C_{\text{annual},0} = \frac{€225{,}600}{3} + €33{,}840 = €109{,}040. \tag{8.2}$$

In subsequent years, the maintenance component increases with inflation, while the initial productization cost remains fixed. For the central cost scenario, the annual cost-recovery requirement increases to €109,886 in FY2027 and €110,753 in FY2028.

Similarly, annual license prices are assumed to increase with inflation according to

$$P_t = P_0(1+i)^t, \tag{8.3}$$

where P_0 is the initial FY2026 license price, i is the annual inflation rate, and t is the number of years after FY2026.

The required number of annual licenses is calculated by dividing the annual cost by the corresponding license price:

$$N_t = \left\lceil \frac{C_{\text{annual},t}}{P_t} \right\rceil, \tag{8.4}$$

where N_t is the required number of customers and the ceiling function ensures that the result is rounded up to the next whole customer.

Table 8.4 shows the number of annual licenses required to cover the annual cost-recovery target for each cost scenario in FY2026 and FY2028. The number of licenses is rounded up to the next whole customer.

Annual license price (FY2026)	Lower cost	Central cost	Upper cost
€5,000	18 / 17	22 / 22	27 / 26
€10,000	9 / 9	11 / 11	14 / 13
€15,000	6 / 6	8 / 8	9 / 9

Table 8.4: Indicative number of annual licenses required to cover annualized productization and maintenance costs. Each cell shows the required licenses in FY2026 / FY2028, with license prices and maintenance costs adjusted annually for 2.5% inflation.

The results show that the business case depends strongly on both the license price and the number of customers. Under the central cost scenario, 11 customers paying €10,000 per year would generate €110,000 in annual recurring revenue, approximately covering the annual cost-recovery target of €109,040.

By FY2028, the corresponding license price would increase to approximately €10,506. With 11 customers, annual recurring revenue would therefore reach approximately €115,569, compared with an annual cost-recovery target of approximately €110,753.

With 20 customers paying €10,000 per year, annual recurring revenue would reach €200,000. Subtracting the annual cost-recovery target would leave €90,960 before sales costs, customer-specific integration, administration, taxes, and other business overhead. This amount is not net profit, but an indication of the potential financial margin before the excluded costs.

This break-even calculation concerns the software business only. It excludes the capital required to purchase or manufacture the robotic platforms. The hardware provider or system integrator would need to assess its own hardware margins, integration expenditure, and deployment costs. The agricultural customer would then evaluate the combined investment in hardware, software, and operation.

The published prices in Table 8.2 provide context for the potential capital requirements of agricultural robotics, but they do not establish the cost of a Swarm Field fleet. The cost of such a fleet depends on the selected platforms, the division of tasks between robots, and the equipment required for each task.

The model indicates a possible route to cost recovery, not a verified revenue forecast. Profitability would depend on market demand, sales cycles, customer retention, support, and integration effort. It also assumes license prices can be adjusted for inflation; otherwise, rising maintenance costs would increase the number of customers required for break-even.

8.7 Potential Economic Value for Agricultural Users

The commercial case for the software provider depends partly on whether the complete robotic system provides sufficient value to agricultural users. Potential benefits include reduced expenditure on crop protection products, less manual monitoring, earlier pest treatment, and reduced crop losses. These benefits depend on the performance of the integrated system and cannot be attributed to the algorithm alone without field validation.

Wageningen Economic Research reported average expenditure on crop protection products of €577 per hectare in Dutch arable farming in 2023 (Wageningen Economic Research, 2025). This figure provides a reference for an illustrative farm-level calculation. It is retained in its original 2023 price basis and is not adjusted for inflation in this illustrative calculation.

For a hypothetical farm of 100 hectares, the corresponding annual expenditure would be

$$C_{\text{protection}} = 100 \times €577 = €57{,}700. \tag{8.5}$$

Not all of this expenditure would necessarily be affected by the swarm. The following calculation therefore assumes that 30% of the expenditure is relevant to treatment decisions that the technology could influence. This proportion is a modeling assumption, not a measured result.

The addressable expenditure is consequently €17,310 per year. Table 8.5 shows the potential gross savings if the integrated system reduces this expenditure by 10%, 20%, or 30%.

The calculation indicates that pesticide savings alone may not justify a high annual software fee for this illustrative farm. At an annual fee of €5,000, a reduction of approximately 29% in the addressable expenditure would be required to recover the software fee through crop protection savings alone.

Reduction in addressable expenditure	Gross annual savings	Annual savings after €5,000 software fee
10%	€1,731	€-3,269
20%	€3,462	€-1,538
30%	€5,193	€193

Table 8.5: Illustrative annual savings for a 100-hectare arable farm, excluding robot and operating costs.

The swarm may provide additional value through reduced monitoring labor, crop losses, and other operational benefits. However, robot acquisition, operation, and maintenance costs are also excluded.

The hardware prices in Table 8.2 provide context, but do not establish the cost of a Swarm Field fleet, which depends on the selected platforms, task allocation, and required equipment.

Rajmis et al. (Rajmis, Karpinski, Kehlenbeck, Pohl, & Herrmann, 2022) reported pesticide-application cost savings of 26–66% in modeled scenarios for site-specific crop treatment in northern Germany. Their analysis included pesticide, labor, machinery, and monitoring costs, and therefore assessed a broader intervention than the Swarm Field algorithm alone. These results demonstrate the potential economic value of targeted crop treatment, but they should not be interpreted as evidence that the developed algorithm would achieve equivalent savings.

The software should therefore be priced according to the value demonstrated for each deployment. Licensing through an integration partner could distribute the cost of the software across multiple robots or farms, rather than requiring a single farm to bear the full cost of commercial software development. A hardware provider could also offer the complete system as a service, potentially spreading the initial investment across multiple users. Such an arrangement would require a separate assessment of fleet utilization, maintenance, and service costs.

8.8 Commercialization Requirements

The commercialization of the Swarm Field algorithm depends on reliable operation on physical robots, demonstrated customer demand, and a clear framework for software ownership and support. These conditions must be addressed before the licensing model can be considered commercially viable.

First, the algorithm must be validated on physical robots. The simulation results provide a basis for evaluating swarm behavior, but they do not establish performance under real field conditions. Integration trials should assess whether the algorithm operates reliably with the positioning systems, communication interfaces, sensors, and robot capabilities available on the target platforms. The results can then be compared with the simulation benchmarks to identify remaining limitations.

Second, the proposed commercial model must be validated with potential customers. Discussions with agricultural robot manufacturers and system integrators should establish whether the algorithm addresses a relevant coordination problem, how much integration effort is required, and whether the expected benefits justify a software license. Feedback from farmers can provide additional evidence of the economic value that the complete robotic system would need to deliver, including the costs of the hardware.

Finally, the responsibilities associated with commercial deployment must be established. This includes clarifying ownership of the software, licensing rights, maintenance obligations, and responsibility for software-related failures. Clear interface specifications and documentation would also help reduce integration effort when the algorithm is transferred to different robotic platforms.

9

Risk, Ethics, and Mitigation Strategies

This chapter discusses the risks of the developed swarm algorithm and its possible impact on society. Section 9.1 assesses the risks for the Robotics Institute and for society, together with the actions that mitigate them. Section 9.2 then weighs the ethical considerations that follow from these risks.

9.1 Risk Management

The project was carried out for the Robotics Institute, a research institute rather than a commercial company. This did not mean that the project was free of risk. The institute's main interest is research that can be continued and eventually deployed on physical robots. The largest risk was therefore that the algorithm performed well in simulation but did not work on actual robots in a real crop field. Besides the risks for the institute, a robot swarm that operates autonomously in agriculture also affects farmers, field workers, and the environment. This section first describes the method used to assess the risks, then the risks for the Robotics Institute, and finally the impact and risks for society.

9.1.1 Risk Assessment Method

The risk assessment followed the process of ISO 31000 (International Organization for Standardization, 2018): the risks were identified, analyzed, evaluated, and treated. The risks were identified in a brainstorm session, using the design choices of the midterm report, the physical capabilities of the hardware provided by the Swarm Head team, and the literature review as input. Each risk was given a likelihood L and an impact I, both on a scale from 1 (very low) to 5 (very high), as defined in Table 9.1. The risk score R was then computed as

$$R = L \cdot I, \tag{9.1}$$

which ranges from 1 to 25 and was used to rank the risks. The level of a risk was read from the risk matrix in Table 9.5, which classifies each combination of likelihood and impact as low, low-medium, medium, medium-high, or high. For every risk of medium level or higher, a mitigating action was defined, after which its residual likelihood and impact were estimated. Table 9.6 shows the resulting levels after mitigation.

Score	Likelihood L	Impact I
1	Very unlikely to occur	Negligible effect on the results or on people
2	Unlikely to occur	Minor delay or minor loss of performance
3	Possible	Noticeable loss of performance or credibility
4	Likely to occur	Main conclusions or a deployment are compromised
5	Almost certain to occur	Project goal fails or people are harmed

Table 9.1: Scales used for the likelihood and the impact of a risk.

9.1.2 Risks for the Robotics Institute

Table 9.2 lists the risks for the Robotics Institute, together with their score before and after mitigation. The risks fell into three groups: technical risks of the swarm itself, risks to the validity of the research, and risks to the continuation of the project.

ID	Risk	L	I	R	Mitigating action	R_{res}
C1	Algorithm works in simulation but not on physical robots	4	5	20	Documented model assumptions, staged hardware tests recommended	12
C2	Positioning is not accurate enough for the potential map	3	5	15	Positioning accuracy specified as a hardware requirement; position-error robustness remains to be validated	8
C3	Radio communication between robots fails	3	3	9	Seeded killers act on their own memory; radio range and robots that went dark tested in simulation	6
C4	Individual robots fail during operation	4	3	12	Robots that went dark or became immobile injected during the run	6
C5	Tuned parameters only perform well in the tested scenarios	3	3	9	Experiments on several fields, swarm sizes and pest rates, over several seeds	6
C6	Algorithm does not match the hardware of the Swarm Head team	3	4	12	Shared interface and agreed hardware parameters	6
C7	Pest model does not reflect real infestations	3	4	12	Pest rate varied between experiments; spreading pests left for future work	8
C8	Knowledge is lost after the project ends	4	3	12	Documentation, automated tests, and a handover to the institute	4
C9	Hardware cost of a large swarm is too high	3	4	12	Heterogeneous swarm with few ground robots, cost estimate in the business case	9

Table 9.2: Risk register for the Robotics Institute, with the risk score R before and the residual score R_{res} after mitigation.

The gap between simulation and reality (C1) received the highest score. The simulation modeled the field as a coarse grid, the pests were static, detection and treatment were perfect, and every message within radio range arrived, so the robots did not face the delays and errors of real sensors and radios. All modeling assumptions were documented so that the institute can identify which ones must be verified on hardware. The risk could not be eliminated within the project because physical robots were not available. Testing the algorithm step by step, first with hardware in the loop and then in a small field trial, is therefore recommended.

Both methods required every robot to know its position (C2): a seeded robot computed its map over absolute cells, and a hop-based beacon marked the cell of the robot that sent it. If the position error became larger than one grid cell, a robot could follow the wrong slope of the potential map or the wrong beacon. The required positioning accuracy should therefore be specified as a hardware requirement. Since the report does not present a direct experiment on position-error robustness, this remains an open validation task for future hardware-in-the-loop and field testing.

Communication and robot failures (C3 and C4) were expected to occur in any real deployment. The seeded killers reduced their impact by design: each killer built its potential map from its own memory, so it did not need an uninterrupted chain of communicating peers to navigate towards a pest. The hop-based killers did need such a chain. The validation tested robots that went dark, which stopped both communicating and working, and robots that became immobile but kept communicating (Section 7.1.8), as well as shorter radio ranges (Section 7.1.5). Robots that went dark hurt the hop-based killers more than the seeded killers, and immobile robots hurt both methods most. Failures of the radio alone, in which a robot kept working but lost or dropped messages, were not tested, and these simulation tests do not establish how the system would behave with real communication hardware or physical failures.

The remaining risks concerned the validity and continuation of the research. The parameters of the arbiter and the potential map were tuned in simulation, which created the risk that they only performed well in the tested scenarios (C5). The experiments therefore varied the field, the swarm size, the pest rate and the radio range, and repeated every configuration over several random seeds. The pest model itself (C7) remained simple: the pests did not grow or spread, so a more realistic pest model is part of

the future work. To prevent knowledge loss after the project (C8), the code was documented, covered by automated tests, and handed over to the Robotics Institute.

9.1.3 Impact and Risks for Society

A robot swarm that monitors and treats crops changes how farming is done, which affects more stakeholders than the Robotics Institute alone. The analysis was structured with the People, Planet, and Prosperity framework of Section 2.3, and was guided by the dimensions of responsible innovation of Stilgoe et al. (Stilgoe, Owen, & Macnaghten, 2013): anticipating impacts and involving the affected stakeholders. Table 9.3 summarizes the expected positive and negative impacts per stakeholder.

Stakeholder	PPP	Positive impact	Negative impact
Farmers	Prosperity	Earlier detection of pests, less yield loss, less monitoring work	High investment, dependence on technology and suppliers
Field workers	People	Less repetitive and physically demanding work	Fewer jobs, need for new skills
Environment	Planet	Targeted treatment reduces pesticide use	Batteries, electronic waste, and energy use
Neighbors and public	People	Less pesticide drift near homes	Privacy concerns from camera drones, safety near people
Society	Prosperity	More resilient food production (SDG 2)	Unequal access between large and small farms

Table 9.3: Expected positive and negative impact of the swarm per stakeholder.

The negative impacts in Table 9.3 were translated into risks and scored with the same method as in Section 9.1.1. Table 9.4 shows the result. Because the project delivered a simulation, most of these risks only arise once the swarm is deployed. The mitigating actions are therefore partly design choices that were already made and partly recommendations for the Robotics Institute.

ID	Risk	L	I	R	Mitigating action	R_{res}
S1	Robot collides with a person or animal	2	5	10	Predictable behavior through the arbiter, safety zones and emergency stop on hardware	5
S2	Wrong treatment: missed pests or unnecessary pesticide use	3	4	12	Farmer confirms treatment decisions, detection uncertainty to be modeled before deployment	6
S3	Loss of jobs for field workers	3	3	9	Swarm positioned as support tool, training for operating and maintaining robots	6
S4	Misuse of field and camera data	3	3	9	Data owned by the farmer, collection limited to the field	4
S5	Only large farms can afford the swarm	4	3	12	Scalable swarm size, shared ownership through cooperatives	9
S6	Environmental cost of batteries and hardware	3	3	9	Life cycle assessment, repairable and reusable robots	6
S7	Unclear liability for decisions of autonomous robots	3	4	12	Human oversight, compliance with drone and machinery regulation	8

Table 9.4: Risk register for society, with the risk score R before and the residual score R_{res} after mitigation.

Safety around people and animals (S1) had the highest possible impact. Swarm behavior emerges from local rules, which makes it harder to predict than the behavior of a single robot. The arbiter with hysteresis contributed to predictability, because a robot kept its choice until another option was clearly better instead of changing direction at every new message. On the hardware side, safety zones around

people and an emergency stop are required before any field trial.

Wrong treatment decisions (S2) affect both the farmer and the environment. A missed pest leads to yield loss, while a false detection leads to unnecessary pesticide use, which works against the environmental benefit that motivated the project. The simulation assumed perfect detection and treatment, so this uncertainty was not yet tested, and modeling it is part of the future work. For a deployment, it is recommended that the farmer confirms treatment decisions until the detection is shown to be reliable.

The social risks (S3 to S5) cannot be solved by the algorithm alone. The swarm reduces repetitive monitoring work, but it also changes the role of field workers and requires new skills. Moreover, the cost of a swarm may favor large farms, which conflicts with the goal of sustainable food production for all. Because the algorithm works for any swarm size, smaller farms could start with a few robots or share a swarm through a cooperative. Finally, regulation and liability (S7) are still developing for autonomous agricultural robots and drones. Human oversight over treatment decisions keeps a person responsible, which also connects the risks to the ethical considerations in Section 9.2.

9.1.4 Risk Matrix

Table 9.5 and Table 9.6 place all risks of Table 9.2 and Table 9.4 in the risk matrix, before and after mitigation. Before mitigation, the gap between simulation and reality (C1) was the only risk at a high level, seven risks were at a medium-high level, and the remaining eight at a medium level. After mitigation, no risk remained at a high level, and nine of the sixteen risks dropped to a low-medium level. C1 remained at medium-high, because it can only be reduced further by tests on physical robots, which were outside the scope of the project. Safety around people (S1) dropped from medium-high to medium, but not further, because its impact stays severe however unlikely a collision becomes.

	Impact I				
Likelihood L	Negligible	Minor	Moderate	Significant	Severe
Very likely					
Likely			C4, C8, S5		C1
Possible			C3, C5, S3, S4, S6	C6, C7, C9, S2, S7	C2
Unlikely					S1
Very unlikely					

Table 9.5: Risk matrix before mitigation, with the risks of the Robotics Institute (C) and of society (S).

	Impact I				
Likelihood L	Negligible	Minor	Moderate	Significant	Severe
Very likely					
Likely					
Possible			C9, S5	C1	
Unlikely		C8, S4	C3, C4, C5, C6, S2, S3, S6	C2, C7, S7	
Very unlikely					S1

Table 9.6: Risk matrix after mitigation, with the risks of the Robotics Institute (C) and of society (S).

9.2 Ethical Considerations

A swarm that monitors a field and decides by itself where to treat raises ethical questions that go beyond the risks of Section 9.1: who stays in control of the decisions, who benefits, and who carries the costs. This section first defines the criteria against which the design was evaluated, then weighs the chosen solution against them, and finally discusses the dilemmas in which these criteria conflicted.

9.2.1 Ethical Criteria

The design was evaluated against the seven requirements of the Ethics Guidelines for Trustworthy AI of the European Commission (High-Level Expert Group on Artificial Intelligence, 2019). These requirements were chosen because they were written for autonomous systems that make decisions affecting people, which is what a robot swarm in agriculture is. The requirements were human agency and oversight, technical robustness and safety, privacy and data governance, transparency, diversity and fairness, societal and environmental well-being, and accountability. For societal and environmental well-being, the analysis additionally used the doughnut model of Raworth (Raworth, 2017), which describes a safe space between a social foundation that every person needs and an ecological ceiling that human activity should not exceed.

9.2.2 Weighing the Design

Table 9.7 shows how the final design related to each requirement. For every requirement, the team identified which design choices supported it, which concerns remained, and to what extent the requirement was met.

Requirement	Supported by	Remaining concern	Met
Human agency and oversight	Time controls and robot inspection in the simulation, farmer confirms treatments (S2)	Robots choose targets autonomously	Partly
Robustness and safety	Decentralized design without a single point of failure, robot failures tested in simulation, arbiter with hysteresis	Only validated in simulation (C1), safety near people (S1)	Partly
Privacy and data governance	Robots share only observations of the field and of each other's position and task, within radio range	Camera data and data ownership after deployment (S4)	Open
Transparency	Each robot reports its goal, target and beliefs, runs can be recorded and replayed	Emergent swarm behavior is hard to explain as a whole	Partly
Diversity and fairness	Algorithm works for any swarm size	Cost favors large farms (S5)	Partly
Societal and environmental well-being	Targeted treatment reduces pesticide use	Batteries, hardware, and changing jobs (S3, S6)	Partly
Accountability	Reproducible runs from a seed, recorded commands	Liability for autonomous decisions (S7)	Open

Table 9.7: The final design weighed against the requirements for trustworthy autonomous systems.

The requirements that the design supported best were robustness and transparency, because these could be addressed in the algorithm itself. The swarm had no central controller, so the failure of one robot did not stop the others, and the arbiter kept a robot on its choice instead of changing direction at every new message, which made its behavior predictable. Every robot could report its current goal, target, and beliefs, and every run could be reproduced exactly from its seed and recorded commands. This made it possible to trace why a robot made a decision, which is a precondition for accountability.

The requirements that remained open were privacy and accountability. These depend on how the swarm is deployed rather than on the algorithm: who owns the data that the robots collect, and who is responsible when an autonomous robot makes a wrong decision. The project could not settle these questions, but the design kept them open for a responsible answer, because the robots shared only observations of the field and of each other's position and task, and every decision could be traced afterwards.

From the perspective of the doughnut model, the swarm was intended to move agriculture away from the ecological ceiling, by reducing the chemical pollution caused by spraying whole fields, while supporting the social foundation of food security. At the same time, the hardware of the swarm adds to resource use and electronic waste, and the automation changes the work of people on the farm. Whether the swarm stays within the doughnut therefore depends on the net reduction of pesticides compared with the material cost of the robots, which is a question for a life cycle assessment after the first field trials.

9.2.3 Ethical Dilemmas

Three dilemmas arose in which two of the criteria conflicted, and for each the team made a deliberate choice.

The first dilemma was autonomy against oversight. A swarm that treats a pest as soon as it detects one reacts fastest, which limits the spread of a pest and the amount of treatment needed. However, a wrong detection then leads directly to an unnecessary treatment, without a person being able to intervene. The team chose to recommend human confirmation of treatment decisions until the detection is shown to be reliable, accepting a slower response in exchange for keeping the farmer in control.

The second dilemma was efficiency against employment. The swarm reduces repetitive monitoring work, which is a benefit for the people who do it now, but it can also replace them. The team chose to position the swarm as a tool that supports the farmer and the field workers rather than replacing them, and recommended that operating and maintaining the robots is part of the deployment plan, so that new work replaces the old.

The third dilemma was data sharing against privacy. The swarm performed better when the robots shared more of what they observed, but data about a field also says something about the farm and its owner. The team limited the sharing to field observations between the robots themselves, and recommended that the farmer remains the owner of all data collected on the farm. Finally, the team recognized that the coordination of a swarm of drones could also be used for purposes other than agriculture, such as surveillance. Because the project only produced an algorithm for crop monitoring, this risk of dual use was judged to be low, but it should be considered when the work is continued.

Taken together, the design supported the requirements that the algorithm could influence, although none of them could be fully met in simulation alone, and left the remaining ones to choices about deployment, for which recommendations were made. The ethical evaluation therefore did not change the chosen concept, but it added conditions under which it should be deployed: human confirmation of treatments, clear data ownership, and a life cycle assessment of the hardware.

10
Interdisciplinary Teamwork

This chapter reflects on how the team worked together during the project. Section 10.1 describes the composition of the team and the disciplines involved. Section 10.2 describes how the work was organized, and Section 10.3 describes the collaboration with the stakeholders outside the team. Finally, Section 10.4 reflects on what went well, what could have been improved, and what the team learned.

10.1 Team Composition

The team consisted of six students with different educational backgrounds, listed in Table 10.1. The project required knowledge of software engineering, robotics, and agronomy. Software engineering was needed to build a simulation that was fast enough to run large batches of experiments. Robotics was needed for the navigation and decision logic of the robots, and agronomy was needed to make the field, the crop rows, and the behavior of pests realistic.

Team member	Background	Main contribution
Amartya Natayu	Bachelor's and Master's in Mechanical Engineering	Defined the robot body and its capabilities, contributed to building the simulation environment, modeled battery behavior, and researched the business case and commercial potential.
Abhronil Chaudhuri	Bachelor's in Mechatronics Engineering and Master's in Systems and Control	Implemented the arbiter and role rebalancing, conducted experiments, and created and contributed to the concepts.
Tycho Grapendaal	Bachelor's in Computer Science, Master's in Data Science and Artificial Intelligence	Implemented the visualization, the experiments, and the metrics, and was the contact person for the coaches for meetings and questions.
Jin Kuo	Bachelor's and Master's in Aerospace Engineering	Implemented the exploration behavior, contributed ideas to the concept, and initiated and organized activities.
Błażej Łytkowski	Bachelor's in Computer Science & Engineering, Master's in Quantum Information Science & Technology	Designed the robot brain architecture, implemented the seeded and hop-based potential fields, and created an experiment runner.
Izzy van der Giessen	Bachelor's in Computer Science & Engineering, Master's in Quantum Information Science & Technology	Implemented the treatment behavior and contributed to the benchmarking.

Table 10.1: Members of the team, their educational background, and their main contribution to the project.

Although the team members came from different backgrounds, some disciplines that the project needed were not represented in the team, most notably agronomy. To close this knowledge gap, the team started the project with a literature review on the size of typical farms, the main crop types, the diseases and infestations that affect them, and the amount of pesticide that is currently used. Later in the project, the team also interviewed a farmer to learn how crop monitoring is currently done and what the farmer would use the technology for.

The different backgrounds also meant that team members approached the same problem from different angles. For example, some team members focused on the hardware constraints and the safety of the robots, while others focused on whether the swarm could be simulated in a feasible way and which software to use for it.

10.2 Work Process

The project was divided into three phases: a research phase with the literature review and the problem statement, a design phase in which three concepts were generated and compared while the implementation was started, and a development phase in which the selected concept was implemented and evaluated. In the design phase, each concept of Chapter 3 resulted from reasoning through different scenarios and identifying what the swarm would need to handle them.

10.2.1 Division of Work

At the start of the design phase, the team agreed on the architecture of the system before any part was built. The system was split into a robot brain, a simulation, and a visualization, which only communicated through a small set of shared interfaces. This division allowed team members to work on different parts in parallel without waiting for each other. The work on the code and the report was distributed at the start of each week and revisited during the regular meeting on Thursday.

10.2.2 Tools and Agreements

The work was coordinated through a shared code repository on GitHub. Each work item was described as an issue and developed on its own branch, which was merged into the main version after a review by another team member. To keep the code consistent across six authors, the team agreed on a common style guide, automatic checks for formatting and types, and a set of automated tests that had to pass before a change was merged.

The team collaborated on writing the report using Overleaf. Similar to the coding process, agreements were made for the report, such as the use of American English, a consistent format for tables and units, and the use of past tense.

The team met twice a week, on Mondays and Thursdays, to discuss the progress, decide on the next steps, and identify where team members needed help. In between these meetings, the team communicated through a Discord server and documented work progress in a logbook.

10.3 Collaboration with Stakeholders

Besides the team itself, the project involved the Robotics Institute as the client, the coaches, and the Swarm Head team, which developed the hardware of the robots. The algorithm developed in this project had to run on that hardware, so the navigation and decision logic had to match its capabilities, such as the radio range, the sensing accuracy, and the energy consumption.

The team shared a WhatsApp group with the Swarm Head team to confirm the hardware requirements and to avoid inconsistencies between the hardware and the simulation. When a topic required a broader discussion, a meeting with both teams was planned. The team met with the coaches almost every week to present its progress and receive feedback.

10.4 Team Reflection

The development of the team followed the four stages of team development: forming, storming, norming, and performing (Tuckman, 1965). Forming took place during the first three weeks, when the team came together, established the goals, conducted a literature review, and brainstormed potential concepts. During the storming stage, disagreements arose among team members due to different approaches to

solving the problem. In the norming phase, the team resolved these disagreements through effective communication, aligned the objectives, and became more familiar with one another's working styles. Finally, during the performing stage, the team leveraged each member's strengths to achieve greater productivity and smoother collaboration, making progress towards the goals set at the beginning of the project.

Section 10.4.1 describes the aspects of teamwork that went well during the project. Following that, Section 10.4.2 identifies the team's areas for improvement and outlines measures that could be taken to address these weaknesses. Finally, Section 10.4.3 summarizes the lessons that the team learned.

10.4.1 What Went Well

Agreeing on the architecture and the shared interfaces early in the project proved valuable. Because each part could be developed and tested on its own, team members with different backgrounds could contribute without having to understand the entire system. The review of every change by another team member spread knowledge of the code over the team, so that the project did not depend on a single person for any part.

Furthermore, the team maintained a positive and supportive working environment in which different opinions were respected and considered. This allowed team members to share their ideas freely. Team members also demonstrated a strong sense of responsibility and initiative, with individuals willingly taking ownership of tasks.

10.4.2 What Could Be Improved

Working on separate branches in parallel also caused merge conflicts when two changes touched the same part of the code, which cost time near the end of the project. Clearer responsibilities for each team member and more frequent coordination would have reduced such overlapping work.

The communication within the team could have been better at the start of the project. Although the Discord server was used for general communication, team members did not always ask for help or request reviews of their code in time. This improved over the course of the project. The team should encourage members to seek assistance and request reviews as soon as needed. Additionally, holding more work sessions instead of working primarily individually would improve communication.

10.4.3 Lessons Learned

The main lesson of the team was that an early agreement on the architecture and its interfaces made interdisciplinary work possible. However, regular communication, code reviews, and integration were equally important to ensure that the individual components worked together effectively.

In addition, the team learned that more frequent and in-depth communication with stakeholders, particularly the coaches, was necessary. More frequent communication would have given the team a clearer understanding of the stakeholders' needs and expectations, including aspects that were initially overlooked and only raised in later discussions. More in-depth discussions would have allowed the coaches to better understand the techniques used and the rationale behind them, which would have enabled them to provide feedback at an earlier stage.

Overall, the team learned that effective communication was essential not only for internal collaboration but also for maintaining alignment with stakeholders throughout the project. In future projects, the team should prioritize regular communication, both within the team and with stakeholders, to identify potential issues earlier, clarify expectations, and incorporate feedback in a timely manner.

11

Conclusion and Recommendations

This project set out to develop a decentralized algorithm with which a swarm of robots monitors a crop field and treats its pests, and to benchmark it in simulation. The swarm consists of drones, the scouts, that search the field, and ground robots, the killers, that drive to the pests and treat them. Every robot runs the same brain and decides only on what it sensed itself and what nearby robots told it. This chapter first draws the conclusions of the project, and then recommends how its results can be used and continued.

11.1 Conclusions

The main research question asked how such a swarm can detect and treat pests efficiently and robustly. The results answer that the robots should share where the pests are, so that every killer can plan its own way there. In this seeded method, every killer plans its route to a known pest on its own map of the field. In the hop-based method, a signal is passed on from robot to robot, and a killer follows it towards the pest without a map. The experiments of Chapter 7, summarized in Section 7.2, showed the seeded method to be better in every setup, with about half the robots for the same service. A seeded killer can act on a pest that it heard of long ago, from a scout that has since flown away, and its map leads it around rivers. A hop-based signal follows the radio instead. It crosses rivers where a killer cannot follow, and it reaches a killer only in a dense swarm of about one robot per radio range. This reverses the ranking of Chapter 4, which put the hop-based concept first. The weakness of that concept lay in how the radio and the terrain interact, which only testing the concepts against each other could reveal.

The seeded swarm used its robots efficiently. It treated a new pest about four times sooner than robots that patrolled the field without communicating, because its killers drove only to the pests. It also scaled well: twice the field needed twice the robots, since no robot has to coordinate the whole field. What limited the swarm was finding the pests rather than treating them, so extra scouts kept helping while extra killers helped less and less. When the robots could switch roles, the swarm kept the field as clean with a third fewer killers. Together, these results give a rule for sizing a swarm. At about one new pest per hectare per hour, it needed one killer and two to three scouts for every $8.4\,\mathrm{ha}$, and about half as many killers again when the robots had to return to a dock to recharge and restock.

Both swarms were only partly robust. Because no robot leads, the seeded swarm kept working when 40 % of its robots failed during a run, with about a third more cells infested. Failed robots that stopped moving but kept communicating did far more harm, and nearly quadrupled the infested cells. Their peers trusted their last reports, and left them pests that they would never reach. A decentralized swarm thus has no single point of failure, but it must notice when the information of a peer has gone stale.

The structure of the brain proved a sound basis. Every activity of a robot, from exploring and treating to recharging and switching roles, bids on the same scale and uses the same navigation, so new activities could be added as new behaviors. The arbiter keeps a robot on its choice until another is clearly better, which kept the robots predictable. Agreeing early on the boundaries between the brain, the simulation and the visualization also let a team of six with different backgrounds work in parallel.

These conclusions rest on simulation alone. The field was a coarse grid, every robot knew its exact position, the pests did not spread, and detection, treatment and radio were perfect. A run simulates all robots in a single thread, so the largest runs took hours, which limited how often each setup could be repeated. The results therefore show which design works and why, but not how well a real swarm will perform. The step to real robots remains the largest risk of the project (Chapter 9).

11.2 Recommendations

The results can serve several parties. The Robotics Institute can use the simulation as a benchmark, in which another swarm algorithm replaces the brain and is measured in the same experiments. The Swarm Head team can run the brain on its robots. For industry, Chapter 8 proposes to license the software to makers of agricultural robots and to system integrators, and finds that 18 to 27 licenses at €5 000 a year would recover its productization cost within three years. For a typical farm of 100 hectares, pesticide savings alone would not pay such a fee, so the value must also come from less monitoring work and lower crop losses. Spot treatment by small robots may pay off most in new ways of farming that grow several crops side by side. In the Netherlands, strip cropping grows crops in narrow alternating strips, and raised the average yield in a multi-year field trial (Juventia & van Apeldoorn, 2024). Pixel cropping goes further and mixes crops in small patches, which is laborious by hand, and robots are being designed for it (Ditzler & Driessen, 2022). Before the software is sold, it must work on real robots, the demand for it must be confirmed with customers, and its ownership must be settled.

Future work should first take the swarm out of simulation. The brain should run on the Swarm Head robots, first in a simulated field with hardware in the loop and then in a small field trial, measured as in this report. In parallel, the simulated world can become more realistic, with spreading pests, imperfect detection and radio messages that get lost. A second direction is trust between the robots. Today a robot accepts what its peers tell it, as long as the news is recent. The experiments showed that a failed robot that keeps its claims can hold back the whole swarm, and a robot that is wrong about its position would spread wrong news in the same way. Every robot shares its position in each message, so the swarm learns where a moved robot is, but not whether a reported position is right. The robots should therefore learn to judge how far a peer can be trusted, for instance from whether it makes progress on its claims. A third direction is a brain that tunes itself. Its rules and parameters were tuned by hand on a few fields, while a learning method could adapt them to new fields, crops and robots. Finally, the swarm can take on more of the work on a farm. On a field of many crops, the robots could recognize and treat the pests of each crop, and also weed or harvest. A farmer could confirm the treatments that the swarm proposes, which would keep a person in control of every treatment.

The code of the project is available on GitHub[1], with its documentation and tests. Every run can be reproduced exactly from its seed, so the experiments of this report can be repeated and extended.

We started this project with the question whether robots can look after a field together, without a leader. We end it with a swarm that does so in simulation, and with a clear view of what it needs to reach a real field. We thank the Robotics Institute, our coaches, the Swarm Head team and the farmer we interviewed for their guidance and their time. We hope that the next team takes the swarm onto the field.

[1] https://github.com/TychoGrapendaal/JIP_Swarm_Field

References

Almtireen, N., & Ryalat, M. (2026, Apr). Robotics in precision agriculture: Task-, platform-, and evaluation-oriented review. *Robotics*, *15*(4), 81. doi: 10.3390/robotics15040081

Chevaleyre, Y. (2004). Theoretical analysis of the multi-agent patrolling problem. In *Proceedings of the ieee/wic/acm international conference on intelligent agent technology (iat 2004)* (pp. 302–308). IEEE. doi: 10.1109/IAT.2004.1342959

Ditzler, L., & Driessen, C. (2022). Automating agroecology: How to design a farming robot without a monocultural mindset? *Journal of Agricultural and Environmental Ethics*, *35*(1), 2. doi: 10.1007/s10806-021-09876-x

Elkington, J. (1997). *Cannibals with forks: The triple bottom line of 21st century business*. Oxford: Capstone.

European Commission. (2020). *A farm to fork strategy for a fair, healthy and environmentally-friendly food system.* COM(2020) 381 final.

European Commission. (2026, May). *Economic forecast for the netherlands.* Retrieved from https://economy-finance.ec.europa.eu/economic-surveillance-eu-member-states/country-pages-including-country-reports/netherlands/economic-forecast-netherlands_en (Spring 2026 forecast, published 21 May 2026)

Graetz, G., & Michaels, G. (2018, Dec). Robots at work. *The Review of Economics and Statistics*, *100*(5), 753–768. doi: 10.1162/rest_a_00754

Grub, K. (2026, Sep). *Rtk gps accuracy: What 1–2 cm actually means in the field.* Retrieved from https://rtkdata.com/blog/rtk-vs-gps-accuracy-2026/?utm_term=what+is+rtk+corrections&utm_campaign=AZ%2B%7C%2BInfo-Campaign&utm_source=adwords&utm_medium=ppc&utm_id=24115856362&hsa_acc=1555843850&hsa_cam=24115856362&hsa_grp=197715949406&hsa_ad=819989538151&hsa_src=g&hsa_tgt=kwd-2550501175367&hsa_kw=what+is+rtk+corrections&hsa_mt=p&hsa_net=adwords&hsa_ver=3&gad_source=1&gad_campaignid=24115856362&gbraid=0AAAAA-Mtq5S8YWtIaP6trMBhm3YLobjy0&gclid=Cj0KCQjw3YvXBhCDARIsAKB-ox8xwpdAdDJhzpB0_0b26nmwihEr8fhXyFh2xbb5Y2wsoC6R4MMlfXMaAvhWEALw_wcB#h-accuracy-at-a-glance (Last assessed:29/10/2026)

High-Level Expert Group on Artificial Intelligence. (2019). *Ethics guidelines for trustworthy AI* (Tech. Rep.). European Commission.

International Organization for Standardization. (2018). *ISO 31000:2018 risk management – guidelines* (Tech. Rep.). International Organization for Standardization.

Jawhar, I., Mohamed, N., Wu, J., & Al-Jaroodi, J. (2018, Nov). Networking of multi-robot systems: Architectures and requirements. *Journal of Sensor and Actuator Networks*, *7*(4), 52. doi: 10.3390/jsan7040052

Juventia, S. D., & van Apeldoorn, D. F. (2024). Strip cropping increases yield and revenue: Multi-year analysis of an organic system in the Netherlands. *Frontiers in Sustainable Food Systems*, *8*, 1452779. doi: 10.3389/fsufs.2024.1452779

Kalkdijk, J. R., Evenhuis, A., & Schepers, H. T. A. M. (n.d.). *Development of spray strategies to control late blight, 2003–2006.* Retrieved from https://edepot.wur.nl/151071 (Applied Plant Research, Wageningen University and Research)

Kitić, G., Krklješ, D., Panić, M., Petes, C., Birgermajer, S., & Crnojević, V. (2022). Agrobot Lala—an autonomous robotic system for real-time, in-field soil sampling, and analysis of nitrates. *Sensors*, *22*(11), 4207. doi: 10.3390/s22114207

Knab. (2026). *Wat verdient een software developer (zzp)? bekijk uurtarief en winst in 2026.* Retrieved from https://bieb.knab.nl/ondernemen/wat-verdient-een

-software-developer-zzp-bekijk-uurtarief-en-winst (Reported average Dutch self-employed software developer rate of EUR 94 per hour, excluding VAT)

Koren, Y., & Borenstein, J. (1991). Potential field methods and their inherent limitations for mobile robot navigation. In *Proceedings of the 1991 ieee international conference on robotics and automation* (pp. 1398–1404).

Leise, A., Singh, M., Cafaro La Menza, N., Knezevic, S., & Jhala, A. (2025). Targeted weed management with precision sprayers in corn and soybean. *Crops & Soils*, *58*(6). Retrieved from https://www.sciencesocieties.org/publications/crops-soils/2025/june/targeted-weed-management-with-precision-sprayers-in-corn-and

Mendelow, A. L. (1981). Environmental scanning—the impact of the stakeholder concept. In *Proceedings of the second international conference on information systems.* Retrieved from https://aisel.aisnet.org/icis1981/20

Milner, E., Sooriyabandara, M., & Hauert, S. (2023). *Swarm performance indicators: Metrics for robustness, fault tolerance, scalability and adaptability.* Retrieved from https://arxiv.org/abs/2311.01944 doi: 10.48550/arXiv.2311.01944

Papadopoulos, G., Arduini, S., Uyar, H., Psiroukis, V., Kasimati, A., & Fountas, S. (2024). Economic and environmental benefits of digital agricultural technologies in crop production: A review. *Smart Agricultural Technology*, *8*, 100441. doi: 10.1016/j.atech.2024.100441

Rajmis, S., Karpinski, I., Kehlenbeck, H., Pohl, J.-P., & Herrmann, M. (2022). Economic potential of site-specific pesticide application scenarios with direct injection and automatic application assistant in northern germany. *Precision Agriculture*, *23*, 2063–2088. doi: 10.1007/s11119-022-09888-1

Raworth, K. (2017). *Doughnut economics: Seven ways to think like a 21st-century economist*. London: Random House Business.

Savary, S., Willocquet, L., Pethybridge, S. J., Esker, P., McRoberts, N., & Nelson, A. (2019). The global burden of pathogens and pests on major food crops. *Nature Ecology & Evolution*, *3*(3), 430–439. doi: 10.1038/s41559-018-0793-y

Shang, L., Pahmeyer, C., Heckelei, T., Rasch, S., & Storm, H. (2023). How much can farmers pay for weeding robots? a monte carlo simulation study. *Precision Agriculture*, *24*, 1712–1737. doi: 10.1007/s11119-023-10015-x

SHARFAEI, S. (2024, May). Robots and employment: A review. *International Labour Review*, *163*(2), 271–293. doi: 10.1111/ilr.12417

Stilgoe, J., Owen, R., & Macnaghten, P. (2013). Developing a framework for responsible innovation. *Research Policy*, *42*(9), 1568–1580. doi: 10.1016/j.respol.2013.05.008

Tuckman, B. W. (1965). Developmental sequence in small groups. *Psychological Bulletin*, *63*(6), 384–399. doi: 10.1037/h0022100

United Nations. (2015). *Transforming our world: The 2030 agenda for sustainable development.* Resolution A/RES/70/1.

Vahdanjoo, M., Gislum, R., & Sørensen, C. A. G. (2023). Operational, economic, and environmental assessment of an agricultural robot in seeding and weeding operations. *AgriEngineering*, *5*(1), 299–324. doi: 10.3390/agriengineering5010020

Vougioukas, S. G. (2019, May). Agricultural robotics. *Annual Review of Control, Robotics, and Autonomous Systems*, *2*(1), 365–392. doi: 10.1146/annurev-control-053018-023617

Wageningen Economic Research. (2025). *Middelengebruik en kosten stijgen in de akkerbouw.* Agrimatie. Retrieved from https://agrimatie.nl/themaResultaat.aspx?indicatorID=2072&sectorID=2233&subpubID=2232&themaID=2275 (Published 6 October 2025)

A

Full Results of the Experiments

Table A.1 lists the time to explore the whole field, t_{50} and t_{90} for every configuration of the exploration experiment in Section 7.1.2, on both fields.

field	method	scouts	explored (min)	t_{50} (min)	t_{90} (min)	coverage
small	exploring	1	21.27 ± 0.07	10.47 ± 0.10	19.10 ± 0.07	1.00 ± 0.00
small	exploring	5	5.37 ± 0.27	2.27 ± 0.04	4.17 ± 0.11	1.00 ± 0.00
small	exploring	10	3.13 ± 0.14	1.17 ± 0.00	2.27 ± 0.04	1.00 ± 0.00
small	exploring	20	1.70 ± 0.03	0.73 ± 0.04	1.33 ± 0.00	1.00 ± 0.00
small	grid search	1	21.97 ± 0.12	10.70 ± 0.06	20.00 ± 0.12	1.00 ± 0.00
small	grid search	5	5.20 ± 0.06	2.50 ± 0.05	4.30 ± 0.03	1.00 ± 0.00
small	grid search	10	3.27 ± 0.04	1.57 ± 0.04	2.53 ± 0.03	1.00 ± 0.00
small	grid search	20	2.17 ± 0.00	1.07 ± 0.04	1.70 ± 0.03	1.00 ± 0.00
large	exploring	1	83.87 ± 0.14	41.70 ± 0.10	75.40 ± 0.11	1.00 ± 0.00
large	exploring	5	19.03 ± 0.30	8.90 ± 0.07	16.37 ± 0.17	1.00 ± 0.00
large	exploring	10	11.13 ± 0.17	4.50 ± 0.05	8.67 ± 0.09	1.00 ± 0.00
large	exploring	20	7.17 ± 0.18	2.27 ± 0.04	5.03 ± 0.08	1.00 ± 0.00
large	grid search	1	86.43 ± 0.21	43.63 ± 0.14	78.53 ± 0.18	1.00 ± 0.00
large	grid search	5	18.47 ± 0.03	9.23 ± 0.07	16.40 ± 0.07	1.00 ± 0.00
large	grid search	10	10.50 ± 0.05	4.93 ± 0.04	8.63 ± 0.03	1.00 ± 0.00
large	grid search	20	6.57 ± 0.07	2.90 ± 0.04	4.90 ± 0.04	1.00 ± 0.00

Table A.1: Minutes until the scouts had sensed every crop cell (explored), half and nine tenths of them (t_{50}, t_{90}), and the coverage when the run stopped, on the small and the large field with rivers; mean ± standard error over 5 seeds. Radio range 100 m, scouts only.

Table A.2 and Table A.3 list the infested cells and the response time for every setup of the number of scouts experiment in Section 7.1.3.

		Seeded killers		Hop-based killers	
Pest rate	Scouts	100 m	50 m	100 m	50 m
Low	1	6.2 ± 0.4	7.2 ± 0.2	15.2 ± 1.6	**18.9 ± 0.8**
	2	4.0 ± 0.3	4.6 ± 0.3	7.5 ± 1.0	12.7 ± 1.6
	5	2.3 ± 0.1	2.6 ± 0.2	4.9 ± 0.5	5.4 ± 0.3
	10	1.8 ± 0.1	2.0 ± 0.1	3.5 ± 0.2	4.5 ± 0.4
	20	**1.4 ± 0.1**	1.5 ± 0.05	2.7 ± 0.4	3.4 ± 0.2
High	1	20.4 ± 1.4	23.4 ± 0.9	62.0 ± 3.3	**71.0 ± 2.2**
	2	13.4 ± 0.4	15.6 ± 0.4	43.8 ± 0.8	52.6 ± 1.8
	5	9.6 ± 0.9	9.9 ± 0.4	34.7 ± 2.0	32.7 ± 2.3
	10	7.1 ± 0.2	8.1 ± 0.5	22.2 ± 1.1	25.2 ± 2.4
	20	**6.2 ± 0.2**	6.4 ± 0.3	18.1 ± 1.9	24.1 ± 2.3

Table A.2: Average number of infested cells during the run (lower is better) for every number of scouts in Section 7.1.3, for 5 seeded or 5 hop-based killers with a radio range of 100 m or 50 m. In green, the fewest infested cells at each pest rate; in red, the most. Mean ± standard error over 5 seeds; small field with a river, 2 h per run; the low pest rate was 20 and the high pest rate 60 new pests per hour.

		Seeded killers		Hop-based killers	
Pest rate	Scouts	100 m	50 m	100 m	50 m
Low	1	12.8 ± 0.6	14.8 ± 0.3	26.7 ± 2.1	29.2 ± 1.3
	2	**8.0 ± 0.3**	**9.2 ± 0.4**	14.3 ± 1.5	20.4 ± 1.6
	5	4.8 ± 0.1	5.5 ± 0.1	10.0 ± 0.7	11.7 ± 0.8
	10	3.4 ± 0.1	3.9 ± 0.1	**6.7 ± 0.2**	**8.2 ± 0.4**
	20	2.9 ± 0.1	3.1 ± 0.1	4.9 ± 0.3	7.0 ± 0.5
High	1	14.3 ± 0.7	16.5 ± 0.4	33.1 ± 1.2	35.3 ± 1.7
	2	**9.0 ± 0.2**	10.5 ± 0.3	26.8 ± 1.0	30.8 ± 1.4
	5	6.6 ± 0.5	**6.9 ± 0.2**	22.7 ± 1.2	21.7 ± 1.1
	10	4.7 ± 0.1	5.4 ± 0.3	14.7 ± 0.4	16.7 ± 1.3
	20	4.2 ± 0.1	4.4 ± 0.2	**12.6 ± 1.0**	**16.6 ± 1.1**

Table A.3: Response time: average minutes from the moment a pest appeared until it was treated, over the treated pests only (lower is better), for 5 seeded or 5 hop-based killers with a radio range of 100 m or 50 m. In green, the fewest scouts that kept the response time below 10 min; in red, the setups that did not reach 10 min even with 20 scouts. Mean $\pm$ standard error over 5 seeds; small field with a river, 2 h per run; the low pest rate was 20 and the high pest rate 60 new pests per hour.